



Scotty App

Proyecto **Intermodular**



Manuela · José Ramón · Iván · José Luís

IES Mutxamel · Curso 2025/2026 · 1º DAW

ABSTRACT

This project developed Scotty App, a web based educational platform designed to help students learn programming through short interactive challenges, progression levels, points, and digital rewards. The platform addressed the common problem of low motivation and poor engagement in traditional, theory-heavy programming lessons by offering a more practical and game-like learning experience. The project was developed using an iterative and incremental methodology supported by Scrum, allowing the team to organize work in stages and validate each part progressively. The main technologies used included HTML5, CSS3, JavaScript, Java, and SQL Server, together with design and coordination tools such as Figma, GitHub, and Trello. The final result was a functional prototype that included a basic local authentication system, an interactive quiz module with JavaScript-based validation, temporary XP progression stored in localStorage, and an administrative desktop application built with JavaFX connected to a MariaDB database, all presented through a responsive interface adapted to different devices.

Índice Contenido:

1. Título del proyecto.....	7
1.1 Nombre del proyecto	7
1.2 Integrantes del grupo	7
1.3 Roles del equipo	7
2. Introducción	8
2.1 Objetivo del proyecto	8
2.2 Justificación del proyecto	8
2.3 Análisis de lo existente (Benchmarking).....	8
3. Requisitos funcionales de la aplicación.....	9
3.1 Descripción del problema	9
3.2 Funcionalidades principales.....	10
3.3 Tipos de usuarios	10
4. Análisis y Diseño	11
4.1 Diagrama de la arquitectura de la aplicación	11
4.2 Diagrama de casos de uso	13
4.3 Diagrama de clases	14
4.4 Modelo de datos	17
4.4.1 Diagrama Entidad-Relación (E-R)	17
4.4.2 Modelo Relacional.....	18
4.4.3 Script SQL de creación de tablas.....	18
5. Metodología de desarrollo	20
5.1 Ciclo de vida del software	20
5.2 Metodología ágil	21
5.3 Planificación y roles del equipo.....	22
6. Codificación	22
6.1 Backend	22
6.1.1 Entorno de programación Backend	22

6.1.2	Lenguajes y herramientas Backend	23
6.1.3	Aspectos relevantes de la implementación Backend	23
6.2	Frontend	24
6.2.1	Entorno de programación Frontend	24
6.2.2	Lenguajes y herramientas Frontend (HTML, CSS y JavaScript)	24
6.2.3	Aspectos relevantes de la implementación Frontend ...	25
6.2.4	Acceso a datos	25
7.	Manual de usuario	26
7.1	Descripción general	26
7.2	Acceso al sistema	27
7.3	Uso de la aplicación	31
8.	Requisitos e instalación	43
8.1	Requisitos del sistema	43
8.2	Instalación y ejecución	43
9.	Plan de negocio	45
9.1	Público objetivo	45
9.2	Utilidad social o empresarial	45
9.3	Costes y beneficios	45
10.	Prevención de riesgos laborales	46
10.1	Identificación de riesgos	46
10.2	Medidas preventivas	47
11.	Integración de los módulos del ciclo en el proyecto	48
11.1	Lenguajes de marcas	48
11.2	Programación	48
11.3	Bases de datos	48
11.4	Entornos de desarrollo	48
11.5	Sistemas Informáticos	49
11.6.	Inglés	49

11.7. IPE I	49
12. Conclusiones	49
12.1 Conclusiones sobre el trabajo realizado	49
12.2 Aprendizaje adquirido	50
12.3 Posibles ampliaciones y mejoras	50
13. Bibliografía y webgrafía	52

Índice Figuras:

[Figura 1 – Diagrama simple de arquitectura](#)

[Figura 2 – Diagrama de arquitectura de la aplicación](#)

[Figura 3 – Diagrama de casos de uso](#)

[Figura 4 – Diagrama de clases JavaFX](#)

[Figura 5 – Diagrama Entidad-Relación \(E-R\)](#)

[Figura 6 – Script SQL Server](#)

[Figura 7 – Landing page](#)

[Figura 8 – Pantalla de registro](#)

[Figura 9 – Pantalla de inicio de sesión](#)

[Figura 10 – Pantalla principal JavaFX](#)

[Figura 11 – Pantalla de inicio post-login](#)

[Figura 12 – Selección de lenguaje](#)

[Figura 13 – Bloques de Java](#)

[Figura 14 – Introducción al bloque](#)

[Figura 15 – Preguntas del quiz](#)

[Figura 16 – Resultado bloque superado](#)

[Figura 17 – Perfil del usuario](#)

[Figura 18 – Tienda de Scotty.App](#)

[Figura 19 – Carrito con compra simulada](#)

[Figura 20 – Pantalla principal JavaFX](#)

[Figura 21 – Gestión de usuarios JavaFX](#)

[Figura 22 – Gestión de productos JavaFX](#)

[Figura 23 – Gestión de pedidos JavaFX](#)

1. Título del proyecto

1.1 Nombre del proyecto

El proyecto se llama Scotty App.

El nombre conecta con conceptos como scout (explorador) y score (puntuación), buscábamos algo corto, fácil de recordar y con una identidad visual reconocible. Además, queríamos un personaje cercano, paciente y siempre dispuesto a ayudar que pudiera guiar a los usuarios por la aplicación, y nos inspiramos en un perro de estilo futurista pero con una estética cartoon amigable. Con todo ello conseguimos definir la marca y el logotipo del proyecto, acompañados de un eslogan que resume la propuesta de valor: Juega, Aprende, Programa.



1.2 Integrantes del grupo

El equipo está formado por cuatro integrantes: José Ramón, el más meticuloso, analítico y preciso del grupo; Iván, creativo y con una ambición constante por crear y crecer; José Luis, con un gran don de gentes, empático y con una capacidad natural para conectar con las personas; y Manuela, organizada, resolutiva y siempre comprometida con el proyecto. Cuatro perfiles distintos que, combinados, han dado forma a un equipo equilibrado y complementario.

1.3 Roles del equipo

La idea es que todos participemos en todas las áreas, así poder experimentar cada una de las fases que conlleva el desarrollo de un proyecto real.

Sin embargo, decidimos asignarnos un rol principal más afín a la personalidad de cada uno:

José Luis, Analista de Negocio: responsable de la investigación de mercado, la justificación del proyecto y el plan de negocio.

José Ramón, Analista de Sistemas: encargado de definir los requisitos funcionales, los perfiles de usuario y el modelado de datos.

Iván, UX/UI Designer: responsable del diseño visual, los mockups y el desarrollo de la landing page.

Manuela, DevOps / Scrum Master: encargada de la arquitectura, la metodología de trabajo, el seguimiento del equipo y la coherencia documental.

2. Introducción

2.1 Objetivo del proyecto

Nuestro objetivo es desarrollar una plataforma web de refuerzo práctico que permita repasar y afianzar lenguajes de programación de forma entretenida. El usuario elige un lenguaje, completa bloques de preguntas y acumula XP según sus aciertos, progresando de nivel conforme avanza. Para definir el objetivo de forma clara y realista utilizamos el enfoque SMART:

- (S) Específico: construir una versión funcional con registro e inicio de sesión, selección de lenguaje, un sistema de quizzes con validación automática y acumulación de XP vinculada al perfil del usuario.
- (M) Medible: la versión funcional estará completa cuando los quizzes validen respuestas correctamente mediante JavaScript, el XP se sume al perfil del usuario, el registro funcione con validaciones y toda la documentación técnica esté entregada.
- (A) Alcanzable: es viable con los conocimientos que hemos adquirido durante el curso y ajustando el alcance al tiempo del que disponemos.
- (R) Relevante: responde a algo que hemos vivido en primera persona: aprender a programar de forma teórica es desmotivador. Scotty App lo convierte en algo práctico, con un sistema de progresión que invita a seguir.
- (T) Temporal: organizado en hitos desde diciembre de 2025 hasta la entrega final el 10 de mayo de 2026.

2.2 Justificación del proyecto

Como estudiantes de programación hemos sentido la necesidad de reforzar lo aprendido en clase y practicar con fluidez, pero desde un punto de vista más ameno: no como algo forzado u obligatorio, sino como algo que motive a volver. Scotty App nace de esa experiencia. Queremos ofrecer una herramienta donde repasar programación se sienta dinámico y con sentido de progreso, sin perder el rigor del contenido. No se trata de trivializar el aprendizaje, sino de hacerlo más atractivo y sostenible en el tiempo.

2.3 Análisis de lo existente (Benchmarking)

El mercado de las plataformas de aprendizaje gamificado ha crecido enormemente en los últimos años. Hay opciones para todos los perfiles y materias, pero cuando buscamos una herramienta gratuita, orientada específicamente a programación y con progresión por niveles propia, el panorama se estrecha bastante. Analizamos dos de las plataformas más populares de quizzes educativos para entender qué ofrecen, qué les falta y dónde encaja Scotty App.

Kahoot! es probablemente la herramienta de quizzes más conocida en el ámbito educativo. Funciona con preguntas en tiempo real, es muy motivadora y fácil de usar. Sin embargo, está pensada para ser dirigida por un profesor en clase, no para el estudio autónomo, y no tiene un sistema de progresión personal ni acumulación de XP por usuario.

Quizizz es una plataforma de quizzes a ritmo propio donde el alumno puede practicar de forma autónoma acumulando puntos. Tiene una gamificación más elaborada que Kahoot! y permite usarla fuera del aula. Su punto débil es que es generalista: cubre todas las materias pero no está especializada en programación ni ofrece una ruta de aprendizaje por lenguaje.

Lo que diferencia a Scotty App es su especialización: está diseñada exclusivamente para repasar programación, con una ruta clara por lenguaje y bloques progresivos que acumulan XP vinculado al perfil del usuario. No pretendemos competir con plataformas con años de desarrollo, pero sí cubrir un hueco concreto: una herramienta gratuita, especializada y con identidad propia para estudiantes de programación.

Criterio	Kahoot!	Quizizz	ScottyApp
Especialización en programación	No	No	Si
Uso autónomo sin profesor	No	Sí	Si
Sistema de XP y niveles propios	No	Parcial	Si
Ruta por lenguaje	No	No	Si
Gratuito	Parcial	Parcial	100% gratuito

3. Requisitos funcionales de la aplicación

3.1 Descripción del problema

Aprender a programar requiere práctica constante. Los estudiantes necesitamos reforzar los conceptos aprendidos en clase, pero las opciones disponibles suelen ser complejas, de pago o demasiado generalistas. La falta de una herramienta gratuita, especializada y con un sistema de progresión que motive a volver hace que muchos dejen de practicar antes de consolidar los conocimientos. Además, en un mercado laboral donde la

programación es cada vez más demandada, I la fluidez identificando conceptos y resolviendo problemas de programación marca la diferencia. No basta con entender la teoría, hay que practicar hasta que salga solo.

3.2 Funcionalidades principales

Nos hemos centrado en lo esencial: que el usuario pueda repasar programación de forma inmediata, acumular XP y ver su progreso en todo momento.

- Registro e inicio de sesión de usuarios.
- Selección de lenguaje de programación.
- Resolución de bloques de quizzes con corrección automática mediante JavaScript.
- Acumulación de XP por respuestas correctas vinculada al perfil del usuario.
- Progresión por niveles según el XP acumulado.
- Consulta del perfil y progreso personal.
- Gestión interna de usuarios, juegos y productos mediante aplicación JavaFX independiente.
- Catálogo visual de productos en la tienda y simulación de proceso de compra.

3.3 Tipos de usuarios

Hemos definido dos perfiles de usuario, cada uno con un nivel de acceso adaptado a su rol dentro de la plataforma:

Alumno: perfil registrado con acceso completo a la plataforma. Puede seleccionar lenguajes, completar bloques de quizzes, acumular XP, progresar por niveles, consultar su perfil y acceder a la tienda.

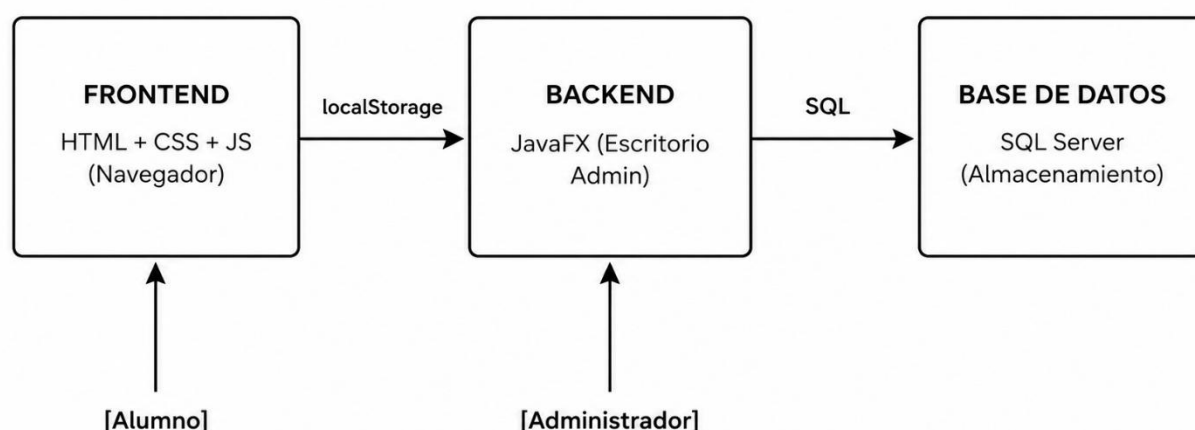
Administrador: gestiona el sistema de forma interna desde la aplicación JavaFX. Se encarga de mantener el contenido actualizado, gestionar las cuentas de usuario y administrar el catálogo de productos de la tienda.

4. Análisis y Diseño

4.1 Diagrama de la arquitectura de la aplicación

Para el desarrollo de Scotty App hemos optado por una estructura separada en tres partes bien diferenciadas: el frontend web, la aplicación administrativa y la base de datos. Esta separación nos permite que cada parte tenga una responsabilidad clara y que el conjunto funcione de forma ordenada.

Figura 1 — Diagrama simple de arquitectura



Frontend (interfaz de usuario). Es la capa con la que el alumno interactúa directamente desde el navegador. Desarrollada con HTML, CSS y JavaScript, se encarga de mostrar el formulario de registro e inicio de sesión, el catálogo de lenguajes, los bloques de quizzes, el perfil del usuario con su XP acumulado y la tienda. La validación de respuestas y la acumulación de XP se gestionan mediante JavaScript, almacenando el progreso de forma temporal en **localStorage** del navegador.

Aplicación administrativa (JavaFX). Es la herramienta de gestión interna del sistema, desarrollada en JavaFX. Se conecta directamente a la base de datos y permite al administrador gestionar usuarios, contenido de los juegos y el catálogo de productos de la tienda. Es de uso exclusivamente interno y los alumnos no tienen acceso a ella.

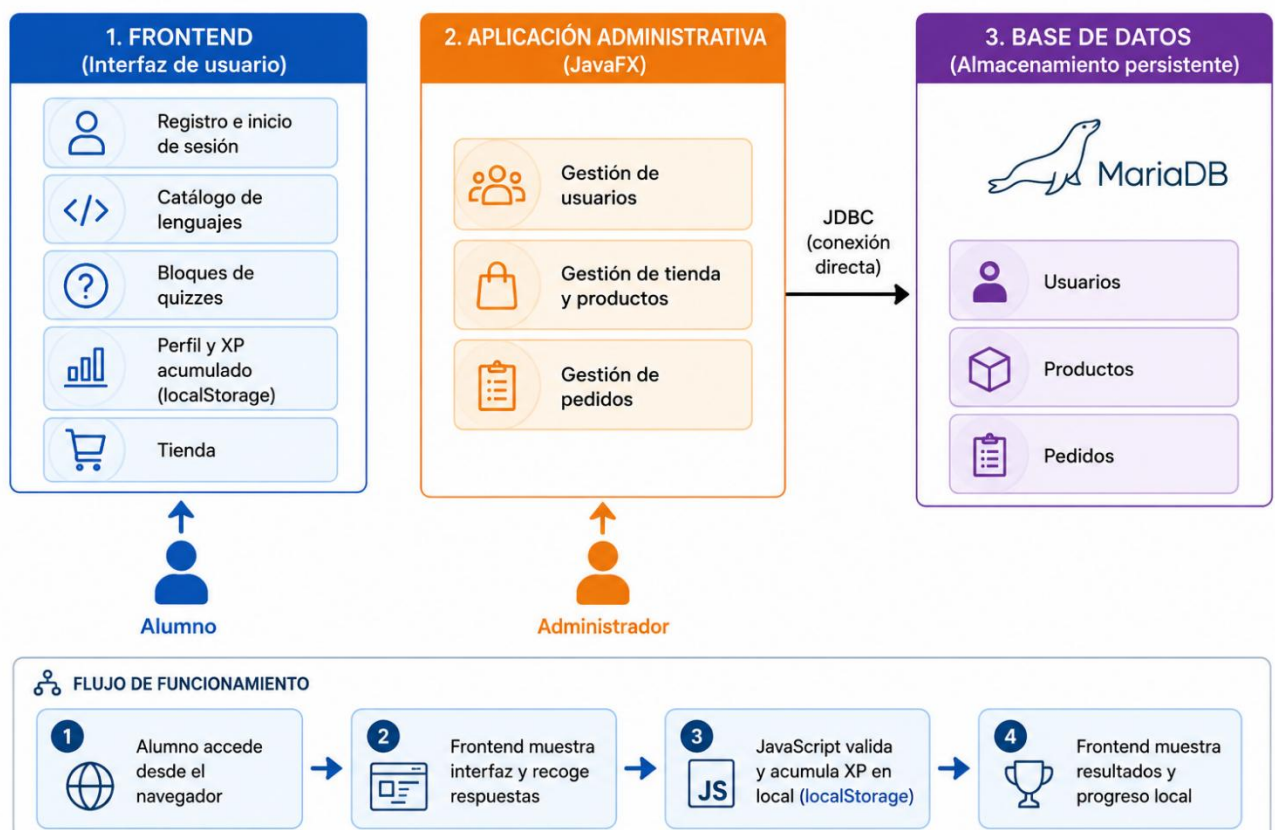
Base de datos (almacenamiento persistente). El diseño y la documentación de la base de datos se han realizado en SQL Server, siguiendo el temario del módulo de Bases de Datos. Para la conexión real con la aplicación JavaFX se utiliza MariaDB, el gestor estudiado en el módulo de Programación. Almacena toda la información de forma persistente: usuarios, productos y pedidos. Actualmente está conectada únicamente a la aplicación JavaFX.

Flujo de funcionamiento:

El alumno accede desde el navegador, el frontend muestra la interfaz y recoge sus respuestas, JavaScript valida las respuestas y actualiza el XP en local, y finalmente el frontend muestra los resultados y el progreso acumulado. El administrador gestiona el sistema desde la aplicación JavaFX, que se conecta directamente a la base de datos.

Como mejora futura, se integrará un backend en Java que conecte el frontend con la base de datos, permitiendo la persistencia real de los datos del alumno.

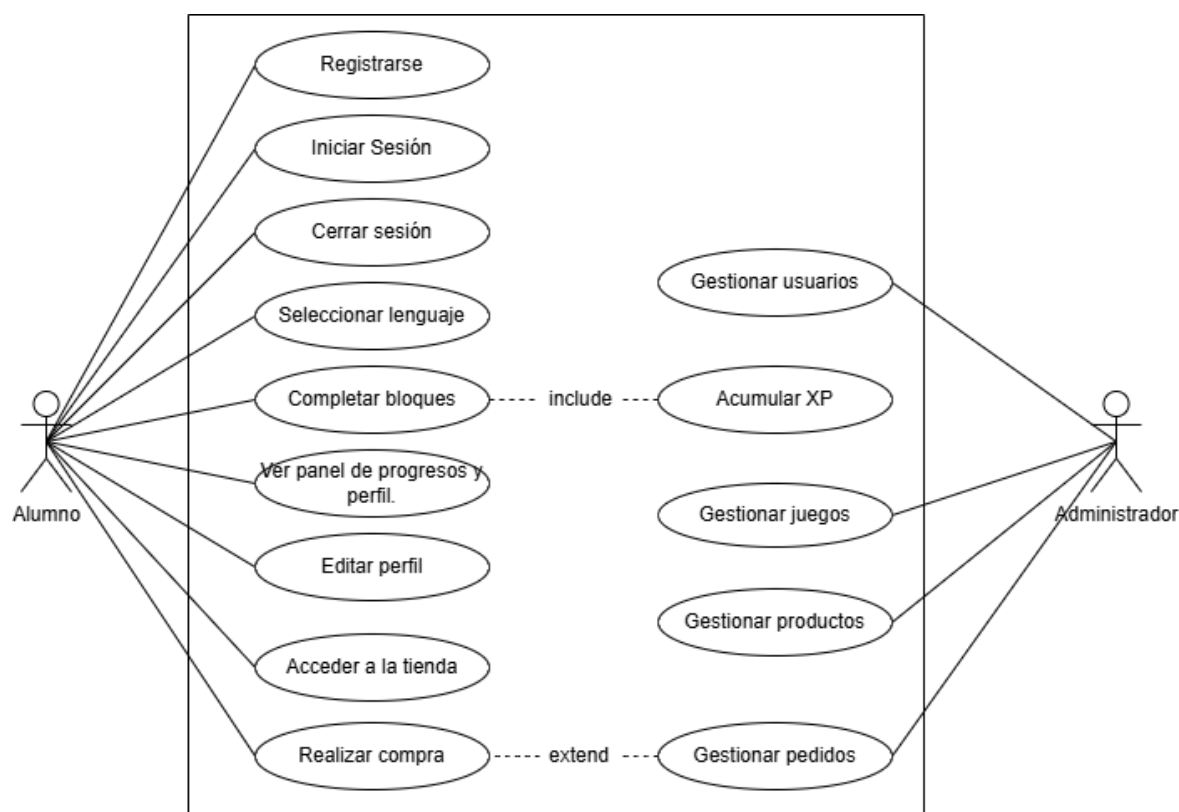
Figura 2 - Diagrama de arquitectura de la aplicación



4.2 Diagrama de casos de uso

Una herramienta que nos ha ayudado a visualizar y aclarar qué hace cada usuario dentro de la plataforma es el diagrama de casos de uso. Gracias a él hemos podido ordenar las ideas y tener claro el alcance de la aplicación antes de ponernos a programar.

Figura 3 — Diagrama de casos de uso



Hemos definido dos actores según el nivel de acceso de cada uno:

Alumno: es el usuario principal de la plataforma. Una vez registrado puede iniciar y cerrar sesión, elegir el lenguaje que quiere practicar, completar bloques de quizzes, consultar y editar su perfil, ver su progreso acumulado y acceder a la tienda para realizar compras.

Administrador: gestiona el sistema de forma global desde la aplicación JavaFX. Se encarga de mantener actualizados los usuarios, los juegos, los productos de la tienda y los pedidos realizados por los alumnos.

Relaciones del diagrama

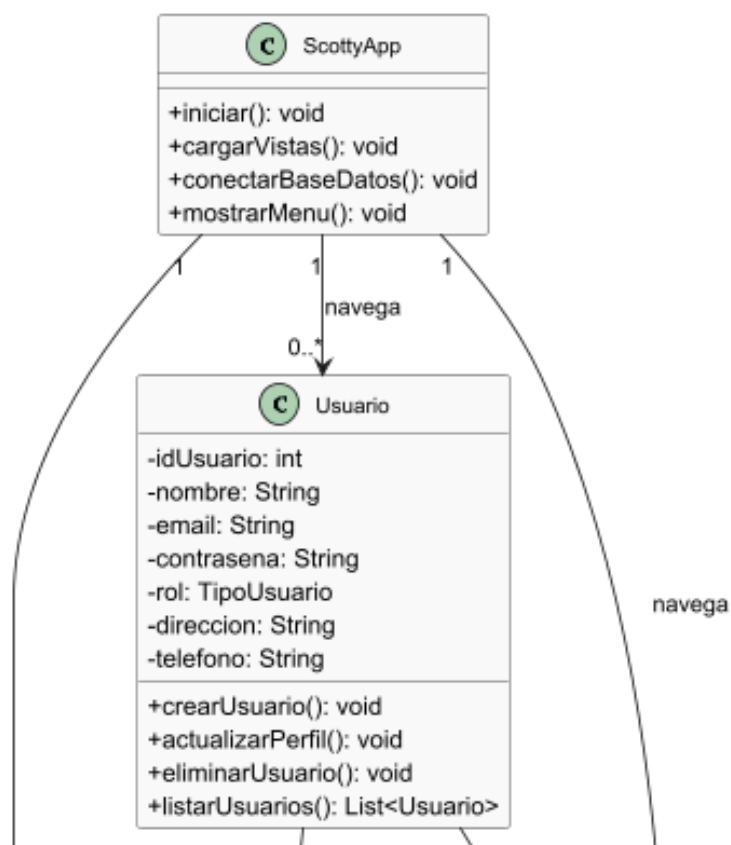
La relación «include» conecta "Completar bloques" con "Acumular XP": siempre que el alumno completa un bloque de quizzes, se le suman puntos de experiencia automáticamente. No puede ocurrir una cosa sin la otra

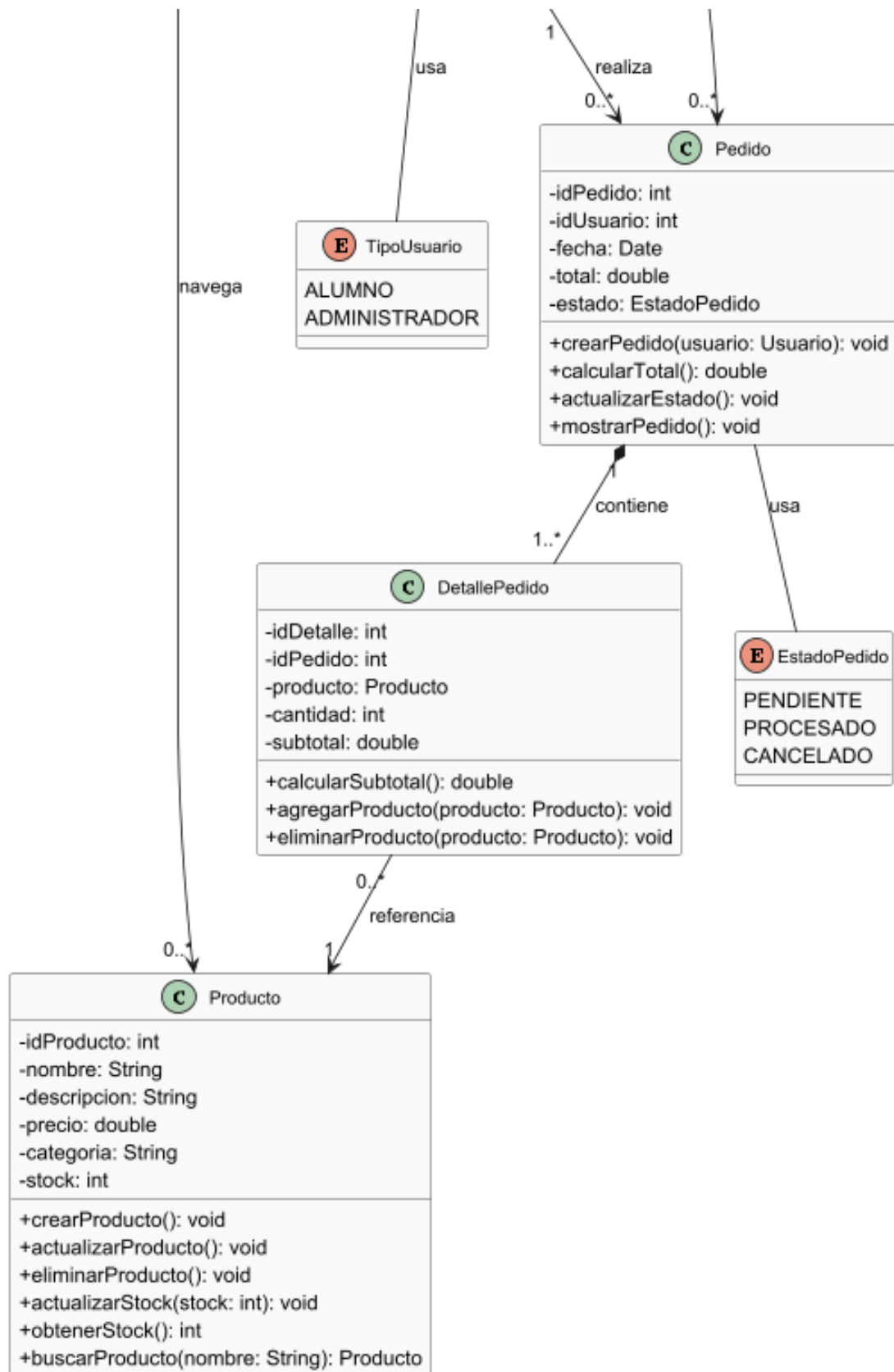
La relación «extend» conecta "Realizar compra" con "Gestionar pedidos": cuando el alumno compra algo en la tienda, ese pedido pasa al administrador para que lo gestione desde JavaFX.

4.3 Diagrama de clases

El diagrama de clases se centra principalmente en la aplicación administrativa desarrollada en JavaFX, ya que la lógica de quizzes, niveles y validación de preguntas se implementa directamente desde el frontend mediante JavaScript.

Figura 4 — Diagrama de clases





- ScottyApp centraliza la navegación de la aplicación administrativa y permite acceder a los distintos módulos del sistema desde un menú principal.
- Usuario almacena los datos de todas las personas registradas en la plataforma: nombre, email, contraseña, dirección y teléfono. El atributo rol utiliza el enum TipoUsuario para distinguir entre ALUMNO y ADMINISTRADOR dentro de la misma entidad.
- Producto representa cada artículo disponible en la tienda. Permite crear, modificar, eliminar y consultar productos, incluyendo el control del stock disponible.
- Pedido registra cada compra realizada por un alumno. Almacena la fecha, el total y el estado del pedido mediante el enum EstadoPedido, que puede ser PENDIENTE, PROCESADO o CANCELADO.
- DetallePedido desglosa el contenido de cada pedido: qué productos se compraron, en qué cantidad y a qué precio. Es necesario como clase independiente porque un pedido puede contener varios productos distintos.
- TipoUsuario es un enum con dos valores: ALUMNO y ADMINISTRADOR, que permite identificar el tipo de acceso de cada usuario de forma clara y sin ambigüedad.
- EstadoPedido es un enum con tres valores: PENDIENTE, PROCESADO y CANCELADO, que garantiza que el estado de un pedido siempre tenga un valor válido y controlado.

Relaciones del diagrama

Main navega hacia Usuario, Producto y Pedido, dando acceso al administrador a cada módulo de gestión desde el menú principal.

Usuario y Pedido se relacionan con cardinalidad uno a muchos: un usuario puede realizar varios pedidos, pero cada pedido pertenece a un único usuario.

Pedido y DetallePedido se relacionan mediante composición: los detalles no existen sin su pedido y desaparecen con él si este se elimina.

DetallePedido referencia a Producto para identificar qué artículo concreto incluye cada línea del pedido.

4.4 Modelo de datos

4.4.1 Diagrama Entidad-Relación (E-R)

Para diseñar la base de datos hemos identificado las entidades principales del sistema y cómo se relacionan entre sí. El diseño del diagrama E-R se ha realizado siguiendo el temario de Bases de Datos con SQL Server como referencia. La base de datos da soporte exclusivamente a la aplicación JavaFX, que gestiona la parte administrativa de la plataforma: usuarios, productos y pedidos.

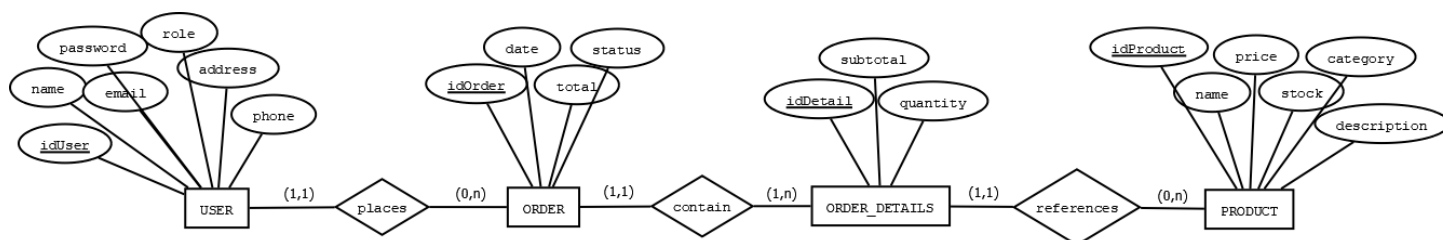
USER centraliza los datos de todos los usuarios registrados: nombre, email, contraseña, rol, dirección y teléfono. El atributo role distingue entre alumno y administrador dentro de la misma entidad, lo que nos permite gestionar ambos perfiles sin necesidad de crear tablas separadas.

ORDER registra cada pedido realizado por un usuario. Para cada pedido guardamos la fecha en que se realizó, el total y el estado actual. Un usuario puede realizar cero o muchos pedidos, pero cada pedido pertenece siempre a un único usuario.

ORDER_DETAILS desglosa el contenido de cada pedido: qué productos se compraron, en qué cantidad y a qué precio. Es necesaria como entidad independiente porque un pedido puede contener varios productos distintos, y necesitamos registrar cada línea por separado.

PRODUCT representa cada artículo disponible en la tienda, almacenando su nombre, descripción, precio, categoría y stock disponible. Un producto puede aparecer en muchos pedidos distintos, pero cada línea de detalle siempre referencia a un único producto.

Figura 5 — Diagrama Entidad-Relación



4.4.2 Modelo Relacional

A partir del diagrama E-R hemos derivado el modelo relacional. Cada entidad se convierte en una tabla con sus atributos, claves primarias (PK) y claves foráneas (FK):

```
USER (idUser, name, email, password, role, address, phone)
ORDER (idOrder, date, total, status, idUser → USER)
ORDER_DETAILS (idDetail, quantity, subtotal, idOrder → ORDER, idProduct → PRODUCT)
PRODUCT (idProduct, name, description, price, category, stock)
```

Este modelo relacional se ha implementado en dos versiones: SQL Server para la documentación y MariaDB para la conexión real con JavaFX.

Hay algunos detalles del modelo que merece la pena comentar:

USER es la tabla central del sistema. Decidimos unificar alumnos y administradores en una sola tabla usando el campo role para distinguirlos, así evitamos duplicar información y la gestión desde JavaFX es mucho más sencilla.

ORDER recoge cada pedido que se realiza en la plataforma. El campo status nos permite saber en todo momento en qué punto está el pedido: pendiente, procesado o cancelado. Incluimos idUser como referencia a USER para saber siempre quién hizo cada pedido, ya que un pedido sin usuario no tendría sentido.

ORDER_DETAILS es la tabla que más trabajo hace, porque desglosa el contenido de cada pedido línea a línea. Para cada línea guardamos cuántas unidades se compraron y el subtotal correspondiente. Apunta tanto al pedido al que pertenece como al producto que se compró, lo que nos permite que un mismo pedido pueda tener tantos productos distintos como necesite sin repetir datos.

PRODUCT almacena el catálogo completo de la tienda con nombre, descripción, precio, categoría y stock disponible. El stock se actualiza desde JavaFX cada vez que el administrador gestiona el inventario o se procesa un nuevo pedido.

4.4.3 Script SQL de creación de tablas

Con el modelo relacional definido, hemos trasladado toda la estructura a SQL Server. Hay algunas decisiones que tomamos durante la implementación que merece la pena comentar:

Hemos implementado dos versiones del script: una para SQL Server, utilizada durante el diseño y la documentación del proyecto, y otra para MariaDB, utilizada en la conexión

real con la aplicación JavaFX. La estructura de tablas es la misma en ambos casos, pero hay diferencias propias de cada gestor: en SQL Server usamos IDENTITY para los identificadores automáticos e LIKE para validar formatos, mientras que en MariaDB usamos AUTO_INCREMENT y REGEXP para una validación más precisa.

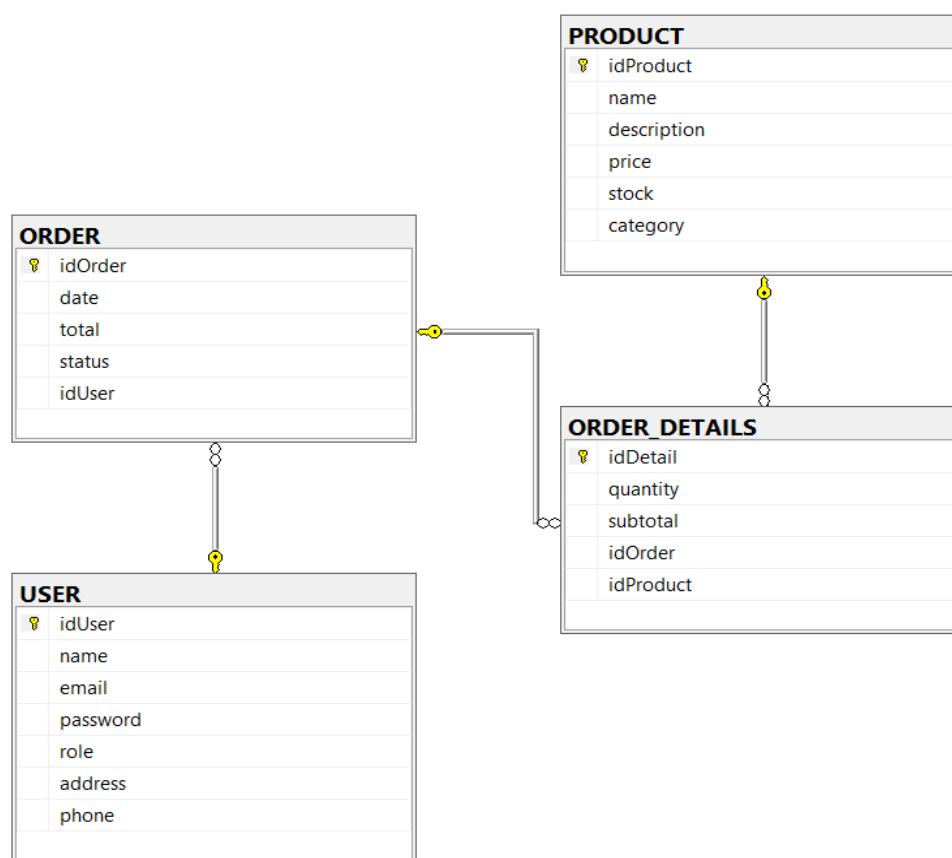
Hemos usado CHECK en los campos que solo admiten valores concretos: el formato del email, el rol del usuario, el estado del pedido, las categorías de producto y el formato del teléfono, definido como char(9) para garantizar exactamente 9 dígitos. Así nos aseguramos de que nadie pueda introducir un valor incorrecto directamente en la base de datos.

Para los campos con valores predecibles hemos definido valores por defecto: role arranca como 'ALUMNI', el status del pedido como 'PENDING', el stock como 0 y la fecha del pedido se rellena automáticamente con el momento de inserción.

Hemos marcado el email como UNIQUE para que no puedan existir dos usuarios con el mismo correo registrado en el sistema.

ORDER y USER van entre corchetes en SQL Server y entre acentos graves en MariaDB porque son palabras reservadas en ambos gestores.

Figura 6 — Script SQL



Además del script de tablas, hemos implementado un trigger que automatiza la actualización del stock. Cuando el administrador cambia el estado de un pedido a 'PROCESSED' desde JavaFX, el stock de cada producto incluido en ese pedido se reduce automáticamente en la base de datos. Sin este trigger, habría que actualizar el stock manualmente cada vez que se procesa un pedido, lo que aumentaría el riesgo de errores.

```
create or alter trigger trg_UpdateStock
on [ORDER]
after update
as
begin
    declare @idorder int=(select idOrder from inserted)
    declare @status varchar(20)=(select status from inserted)

    declare @idproduct int
    declare @quantity int

    if @status='PROCESSED'
    begin
        declare cursor_stock cursor for
            select idProduct, quantity from ORDER_DETAILS
            where idOrder=@idorder

        open cursor_stock
        fetch next from cursor_stock into @idproduct, @quantity

        while @@FETCH_STATUS=0
        begin
            update PRODUCT
            set stock=stock-@quantity
            where idProduct=@idproduct

            fetch next from cursor_stock into @idproduct, @quantity
        end

        close cursor_stock
        deallocate cursor_stock
    end
end
```

5. Metodología de desarrollo

5.1 Ciclo de vida del software

- Hemos analizado los distintos modelos de ciclo de vida para adaptar el que mejor encajara con nuestro proyecto, desde que surgió la idea hasta tener el producto en funcionamiento. Las fases que seguimos, y que además coinciden directamente con los hitos de entrega, fueron las siguientes:
- Análisis: definimos qué tiene que hacer la plataforma, quiénes la van a usar y qué problema resuelve. Partimos de nuestra propia experiencia como estudiantes, tal y como recogemos en el apartado de justificación, y de ahí surgieron los requisitos funcionales, los perfiles de usuario y el benchmarking de plataformas similares.

- **Diseño:** con las ideas claras, pasamos a plasmarlas. Definimos cómo queríamos estructurar la aplicación y qué pantallas necesitábamos, y a partir de esa descripción generamos los mockups con apoyo de inteligencia artificial. Diseñamos cuatro pantallas principales: inicio de sesión, creación de cuenta, selección de juego y bloques de quizzes. Con esos mockups como referencia, elaboramos también la arquitectura, los diagramas de casos de uso y de clases, y el modelo de datos completo.
- **Implementación:** una vez definido el diseño nos pusimos manos a la obra. Usamos Trello para distribuir las tareas y el chat del equipo para coordinarnos en tiempo real. Entre todos construimos el frontend con HTML, CSS y JavaScript, la aplicación administrativa en JavaFX y el diseño de la base de datos en SQL Server y su implementación real en MariaDB para la conexión con JavaFX.
- **Pruebas:** las pruebas actuales son manuales: cada vez que se completa una funcionalidad se comunica al equipo a través del chat y el Trello para que los compañeros validen que todo funciona correctamente. Además, hemos incorporado pruebas con JUnit en la integración de JavaFX, aplicando técnicas de caja negra y análisis de valores límite especialmente en los formularios y la lógica de validación.
- **Despliegue:** el frontend está publicado en CDMON y accesible desde el navegador. La aplicación JavaFX se ejecuta de forma local desde el archivo .jar y está conectada a la base de datos MariaDB.
- **Mantenimiento:** contemplamos la posibilidad de corregir errores y añadir mejoras tras la entrega.

5.2 Metodología ágil

El proyecto estaba estructurado en ciclos cortos con objetivos concretos y roles definidos, por lo que nos encajó perfectamente aplicar Scrum, ya que nos permitía avanzar de forma ordenada sin perder flexibilidad.

Aprovechamos los hitos como nuestros ciclos de trabajo: al inicio de cada uno distribuíamos las tareas en clase, a esto lo llamábamos HitosPlanningClass, y si alguien terminaba antes se ofrecía a ayudar al resto a través de Discord. Conforme se acercaba la entrega final las reuniones se volvieron más frecuentes: cada semana poníamos en común en qué estaba trabajando cada uno, si había algún bloqueo y qué venía después. Al cerrar cada hito reflexionábamos sobre qué había funcionado y qué mejorar, lo que nos permitió detectar problemas y adaptarnos.

5.3 Planificación y roles del equipo

Los roles del equipo están definidos en el punto 1.3, donde cada uno asumió un área principal acorde a su perfil, aunque todos participamos en todas las fases del desarrollo. Para coordinar el trabajo nos hemos apoyado en Trello y GitHub como herramientas principales.

En Trello organizamos el trabajo con un tablero Kanban de cinco columnas: *To Do*, *In Progress*, *Ready for Testing*, *Test* y *Done*. Las tareas se asignaban al inicio de cada hito, cada miembro era responsable de mover sus tarjetas según avanzaba y establecimos un límite de dos tareas activas por persona en *In Progress* para evitar la sobrecarga. El tablero está disponible en: <https://trello.com/b/sttwpXvy/scotty>

Para el control de versiones usamos GitHub bajo la organización Scotty-App (<https://github.com/Scotty-App>), organizado en cuatro ramas: documentación, frontend, backend y base de datos. Cada miembro subía sus avances en su rama y en las reuniones unificábamos versiones quedándonos con la más completa y estable. Los commits se realizan con mensajes descriptivos que indican claramente qué cambio se ha realizado, manteniendo un historial progresivo que refleja la evolución real del proyecto. SourceTree nos permitió gestionar todo esto de forma visual sin depender de la línea de comandos, y el control del repositorio fue una responsabilidad rotativa entre todos.

En la raíz del repositorio hemos incluido un archivo README.md que actúa como carta de presentación del proyecto: nombre del equipo, descripción, integrantes y stack tecnológico.

6. Codificación

6.1 Backend

6.1.1 Entorno de programación Backend

Para desarrollar la aplicación administrativa hemos trabajado en Windows con IntelliJ IDEA como editor principal, ya que tiene soporte nativo para Java y JavaFX y nos facilita mucho el trabajo con autocompletado y detección de errores. Para diseñar las pantallas visualmente hemos usado SceneBuilder, que nos permite arrastrar y soltar componentes sin tener que escribir el XML a mano. Las dependencias del proyecto las gestiona Maven automáticamente, incluyendo el driver de conexión con MariaDB. Para el control de versiones usamos GitHub y SourceTree.

6.1.2 Lenguajes y herramientas Backend

Java es el lenguaje principal de la aplicación administrativa. Lo elegimos porque es el lenguaje que hemos trabajado durante el curso y porque encaja perfectamente con JavaFX y la conexión a base de datos.

JavaFX es el framework que usamos para construir la interfaz gráfica. Cada pantalla se define en un archivo FXML que separa el diseño de la lógica, lo que hace el código más limpio y organizado.

MariaDB es la base de datos a la que conecta la aplicación. La conexión se hace mediante el driver JDBC de MariaDB, que va incluido como dependencia en el proyecto y nos permite lanzar consultas SQL directamente desde Java.

HeidiSQL es la herramienta que usamos para gestionar la base de datos visualmente: crear tablas, revisar datos y ejecutar scripts.

6.1.3 Aspectos relevantes de la implementación Backend

Hemos organizado el código siguiendo el patrón MVC, que separa claramente tres responsabilidades:

- La clase modelo define qué es cada cosa: un usuario, un producto, un pedido.
- La clase controlador gestiona lo que pasa en la pantalla cuando el administrador interactúa.
- La clase mantenimiento se encarga de hablar con la base de datos: consultar, insertar, actualizar y eliminar.

La conexión con MariaDB se abre cada vez que se necesita y se cierra al terminar. Aquí se puede ver cómo se establece:

```
public static Connection conexion() {
    String host = "jdbc:mariadb://localhost:3307/";
    String user = "root";
    String psw = "";
    String bd = "bd_scottyapp";
    try {
        return DriverManager.getConnection(host + bd, user, psw);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```


Las consultas a la base de datos se hacen con Statement y los resultados se recogen en un ResultSet. Por ejemplo, así consultamos todos los usuarios:

```
public static ObservableList<Usuario> consulta(Connection conexion) {
    ObservableList<Usuario> lista = FXCollections.observableArrayList();
    String query = "SELECT * FROM `user`";
    Statement stmt = conexion.createStatement();
    ResultSet respuesta = stmt.executeQuery(query);
    while (respuesta.next()) {
        lista.add(new Usuario(
            respuesta.getInt("idUser"),
            respuesta.getString("name"),
            respuesta.getString("email"),
            respuesta.getString("password"),
            respuesta.getString("role"),
            respuesta.getString("address"),
            respuesta.getString("phone")
        ));
    }
    return lista;
}
```

El resultado se devuelve como ObservableList porque JavaFX lo necesita en ese formato para mostrarlo en tablas y listas de forma automática.

6.2 Frontend

6.2.1 Entorno de programación Frontend

El frontend lo hemos desarrollado con archivos HTML, CSS y JavaScript editados desde IntelliJ IDEA y probados directamente en el navegador. Una vez que todo funciona en local, subimos el código a GitHub con SourceTree y lo desplegamos en el hosting CDMON, donde está accesible desde cualquier navegador.

6.2.2 Lenguajes y herramientas Frontend (HTML, CSS y JavaScript)

HTML5 estructura el contenido de cada página usando etiquetas semánticas como <header>, <nav>, <section> y <form>, lo que hace el código más legible y accesible.

CSS3 define todo el aspecto visual: colores, tipografía, layout y adaptación a distintos tamaños de pantalla.

JavaScript es el que hace que todo funcione. Se encarga de los quizzes, las validaciones de formularios, el perfil del usuario y el carrito de la tienda. Todo ocurre en el navegador del usuario, sin necesidad de servidor.

6.2.3 Aspectos relevantes de la implementación Frontend

Registro y autenticación local

Cuando el usuario se registra, sus datos se guardan en localStorage del navegador como un objeto JSON. Cuando inicia sesión, se recuperan esos datos y se comparan con lo que ha introducido:

```
// Guardar datos al registrarse
localStorage.setItem("usuarioRegistrado", JSON.stringify(usuario));
// Comprobar credenciales al hacer login
const usuarioGuardado = JSON.parse(localStorage.getItem("usuarioRegistrado"));
if (usuarioGuardado.correo === correo && usuarioGuardado.contrasena === contrasena) {
    window.location.href = "../inicio.html";
}
```

Es una solución funcional para el prototipo. En una versión real, las contraseñas se gestionarían en el servidor con cifrado.

Sistema de quizzes y XP

Cuando el alumno termina un bloque, el código cuenta cuántas respuestas ha acertado. Si supera el mínimo, suma XP al perfil. Para que no se pueda acumular XP varias veces en el mismo bloque, guardamos una marca en localStorage:

```
if (aciertos >= 3 && !localStorage.getItem("xpSumada" + clave)) {
const xpActual = Number(localStorage.getItem("xpUsuario")) || 0;
    localStorage.setItem("xpUsuario", xpActual + xpGanada);
    localStorage.setItem("xpSumada" + clave, "true");
}
```

Los puntos varían según el nivel del bloque: 20 XP para los primeros, 30 para los intermedios y 50 para los avanzados.

Carrito de la tienda

La tienda gestiona el carrito en memoria durante la sesión. Los productos se añaden y eliminan dinámicamente y el total se actualiza en tiempo real. La compra es una simulación: al finalizar muestra un mensaje de confirmación y vacía el carrito. No hay pasarela de pago real en esta versión.

6.2.4 Acceso a datos

Al no tener un backend web conectado, cada parte gestiona sus datos de forma independiente. El frontend usa localStorage para guardar temporalmente los datos del

usuario, el XP y el progreso por bloque. Estos datos se mantienen mientras el usuario no borre el almacenamiento del navegador, pero no se comparten entre dispositivos.

La aplicación JavaFX accede directamente a MariaDB mediante JDBC para gestionar usuarios, productos y pedidos de forma persistente.

Como mejora futura, conectaremos ambas partes a través de un backend en Java con una API REST, lo que permitirá que el progreso del alumno se guarde de verdad en la base de datos.

7. Manual de usuario

7.1 Descripción general

Scotty App es una plataforma web de refuerzo práctico para programación. El alumno accede desde el navegador, elige un lenguaje de programación y completa bloques de quizzes acumulando XP en su perfil. Además, dispone de una tienda donde puede consultar y comprar productos de informática y merchandising de Scotty.App. La gestión interna de la plataforma se realiza desde una aplicación de escritorio desarrollada en JavaFX, destinada exclusivamente al administrador, desde la que puede gestionar usuarios, productos y pedidos.

7.2 Acceso al sistema

Plataforma web:

Para acceder a la plataforma el alumno necesita registrarse. En la landing page hay dos botones: "Empieza gratis", que lleva al formulario de registro, y "Ya tengo cuenta", que lleva al inicio de sesión.

Figura 7 — Landing page



En el formulario de registro hay que rellenar nombre, apellidos, nombre de usuario, correo electrónico y contraseña. Al pulsar "Crear cuenta" se accede directamente a la plataforma.

Figura 8 — Pantalla Registro

Scotty.App Iniciar sesión

Crear cuenta

Regístrate para empezar tus quiz, ganar XP y acceder a la tienda.

Nombre

Apellidos

Nombre de usuario

Correo electrónico

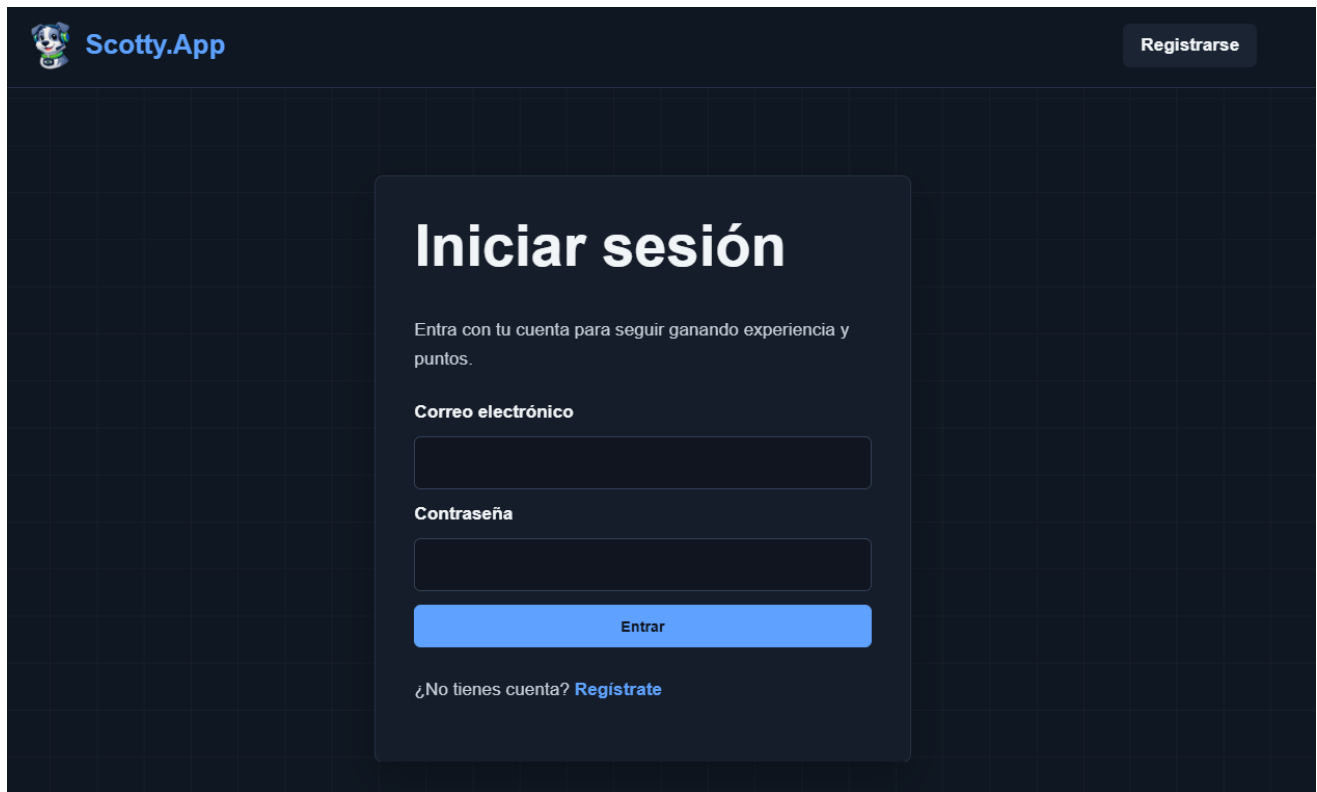
Contraseña

[Crear cuenta](#)

¿Ya tienes cuenta? [Inicia sesión](#)

Si ya se tiene cuenta, basta con introducir el correo y la contraseña en la pantalla de inicio de sesión y pulsar "Entrar". Desde esa misma pantalla hay un enlace directo al registro para los que aún no tienen cuenta.

Figura 9 — Pantalla Inicio



Scotty.App

Registrarse

Iniciar sesión

Entra con tu cuenta para seguir ganando experiencia y puntos.

Correo electrónico

Contraseña

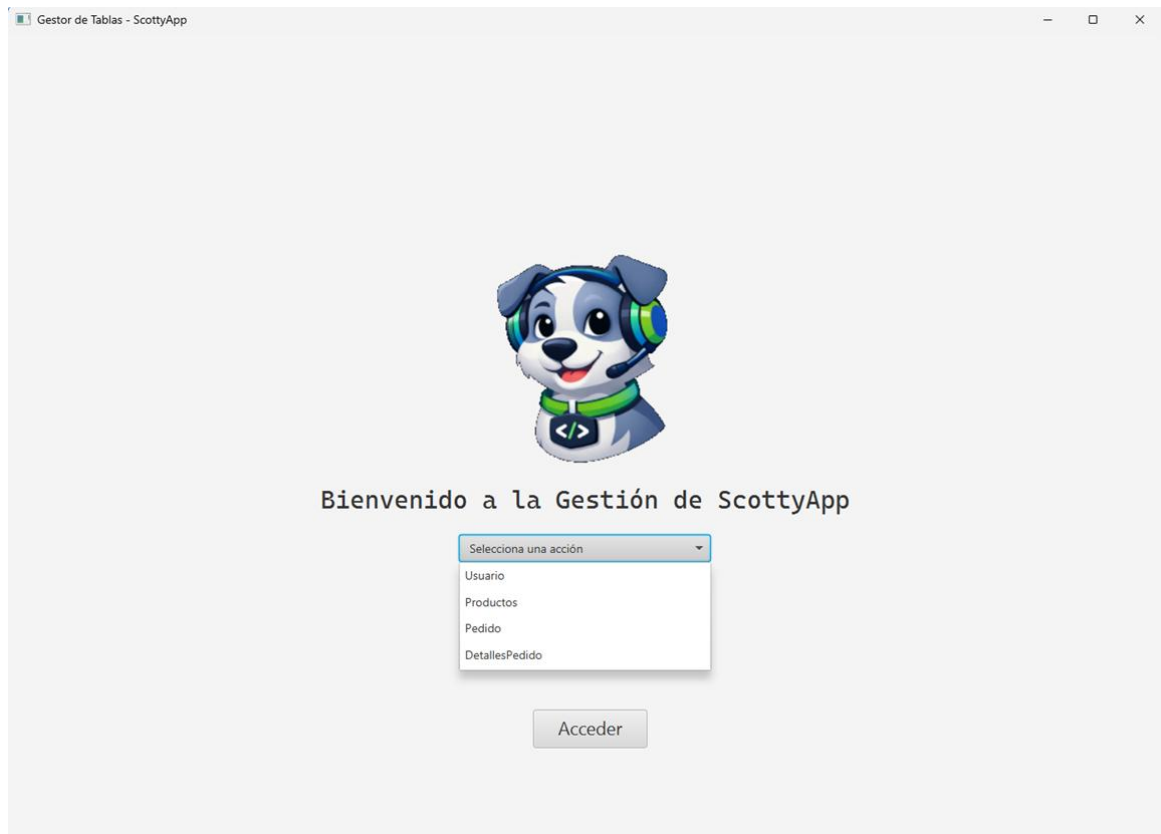
Entrar

¿No tienes cuenta? [Regístrate](#)

Aplicación administrativa JavaFX

El administrador abre la aplicación haciendo doble clic en el acceso directo del escritorio. Al arrancar aparece la pantalla de bienvenida con la mascota de Scotty.App, un menú desplegable con las cuatro opciones de gestión disponibles y el botón "Acceder" para entrar al módulo seleccionado.

Figura 10 — Pantalla principal JavaFX

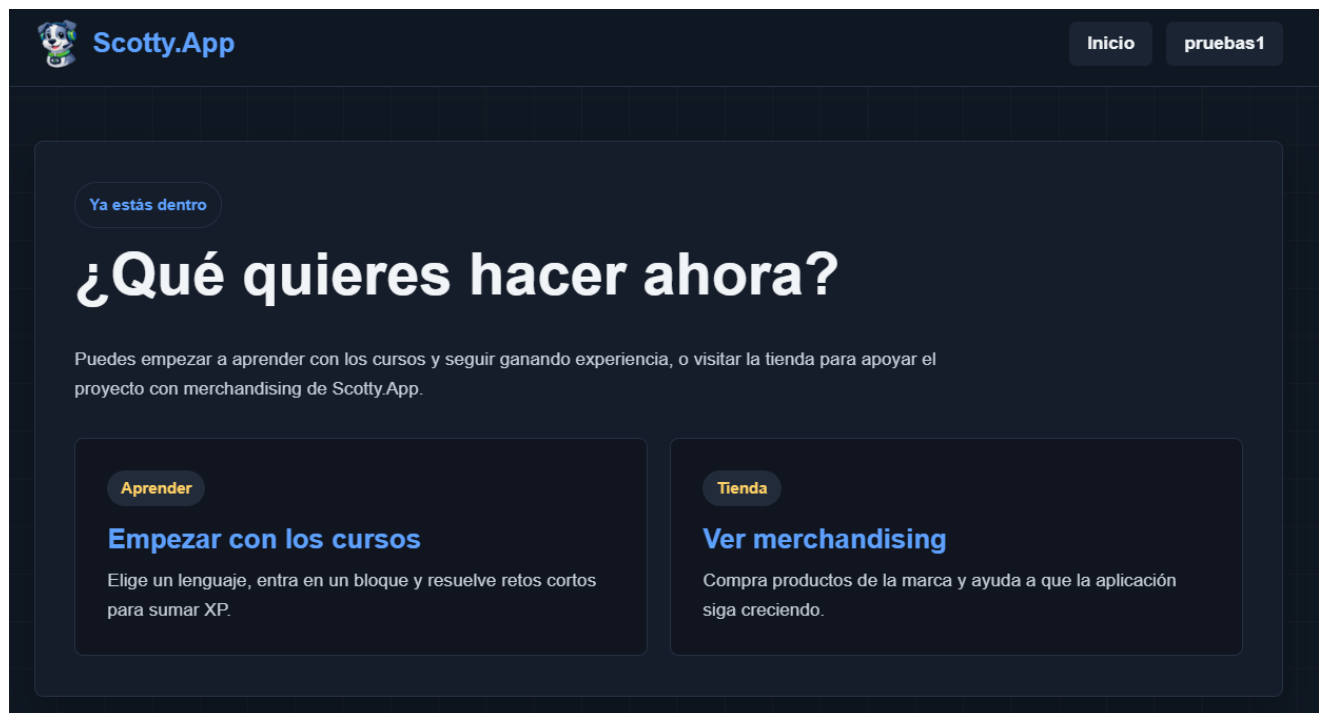


7.3 Uso de la aplicación

Plataforma web.

Una vez dentro, el alumno ve una pantalla con dos opciones: "Empezar con los cursos" para ir a los quizzes, o "Ver merchandising" para ir a la tienda. En la parte superior derecha aparece el nombre de usuario activo.

Figura 11 — Pantalla de inicio post-login



- Quizzes

Al elegir "Empezar con los cursos" aparece el catálogo de los cuatro lenguajes disponibles: Java, JavaScript, SQL y HTML y CSS, cada uno con su icono identificativo.

Figura 12 — Selección de lenguaje



Al seleccionar un lenguaje se muestran todos sus bloques organizados en tres niveles de dificultad: Fácil, Intermedio y Difícil. Cada bloque indica el rango de ejercicios que contiene y su estado actual: Pendiente o Completado. Los bloques ya superados muestran la etiqueta verde "Completado" y el botón cambia a "Repetir bloque".

Figura 13 — Bloques de Java

The screenshot displays the Scotty.App interface for Java exercises. The top navigation bar includes the Scotty.App logo, a 'Inicio' button, and a 'pruebas1' button. A 'Volver' button is located at the top left of the main content area. The main heading is 'Ejercicios de Java', followed by a brief description: 'Elige un bloque y empieza a resolver preguntas de Java. Cada bloque contiene ejercicios cortos para practicar, ganar experiencia y sumar puntos.'

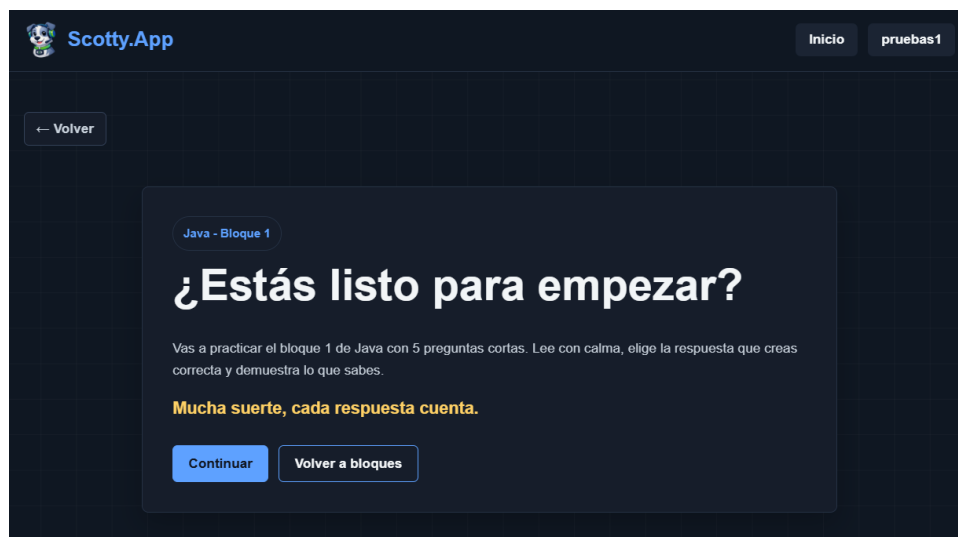
The exercises are organized into three difficulty levels:

- Nivel: Fácil** (Repasa clases, variables, condiciones y bucles sencillos.)
 - Bloque 1**: Ejercicios 1 - 5. Estado: **Completado**. Botón: Repetir bloque.
 - Bloque 2**: Ejercicios 6 - 10. Estado: **Pendiente**. Botón: Empezar bloque.
- Nivel: Intermedio** (Avanza con métodos, arrays, clases y objetos.)
 - Bloque 3**: Ejercicios 11 - 15. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 4**: Ejercicios 16 - 20. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 5**: Ejercicios 21 - 25. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 6**: Ejercicios 26 - 30. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 7**: Ejercicios 31 - 35. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 8**: Ejercicios 36 - 40. Estado: **Pendiente**. Botón: Empezar bloque.
- Nivel: Difícil** (Practica herencia, excepciones, colecciones y buenas costumbres.)
 - Bloque 9**: Ejercicios 41 - 45. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 10**: Ejercicios 46 - 50. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 11**: Ejercicios 51 - 55. Estado: **Pendiente**. Botón: Empezar bloque.
 - Bloque 12**: Ejercicios 56 - 60. Estado: **Pendiente**. Botón: Empezar bloque.

A red arrow labeled 'Estado del Bloque' points from the 'Pendiente' status of Bloque 2 to the 'Completado' status of Bloque 1.

Al entrar en un bloque pendiente aparece una pantalla de introducción que indica el lenguaje, el número de bloque y las instrucciones. Al pulsar "Continuar" comienzan las preguntas. Si se prefiere no empezar, "Volver a bloques" regresa al listado.

Figura 14 — Introducción al bloque



El quiz muestra 5 preguntas tipo test, cada una con tres opciones de respuesta. Hay que responderlas todas y pulsar "Terminar bloque" al final para enviar las respuestas.

Figura 15 — Preguntas del quiz

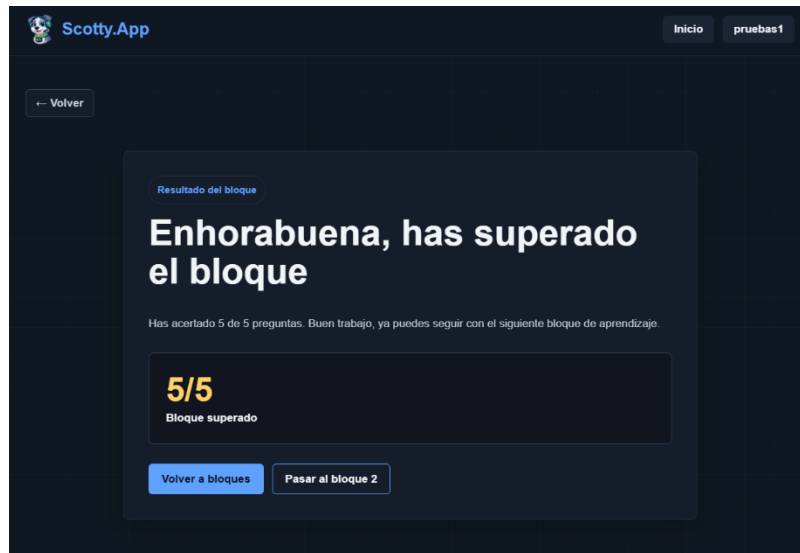
The screenshot shows a web application interface for a Java quiz. At the top, there's a header with the Scotty.App logo and navigation links for 'Inicio' and 'pruebas 1'. Below the header, a 'Volver' button is visible. The main content area is titled 'Java - Bloque 1' and includes a sub-header 'Responde en las preguntas básicas sobre Java y suma los primeros puntos.' The quiz consists of five exercises, each with a question and three multiple-choice options:

- Ejercicio 1:** ¿Qué método se ejecuta primero en una aplicación Java?
 - start()
 - main()
 - init()
- Ejercicio 2:** ¿Qué palabra se usa para crear una clase?
 - class
 - function
 - object
- Ejercicio 3:** ¿Qué tipo de dato guarda números enteros?
 - String
 - int
 - boolean
- Ejercicio 4:** ¿Cómo se muestra texto por consola?
 - console.log()
 - print.text()
 - System.out.println()
- Ejercicio 5:** ¿Qué valor puede tener un boolean?
 - true o false
 - rojo o azul
 - uno o muchos

At the bottom of the quiz area, there is a blue button labeled 'Terminar bloque'.

Si se acierta un mínimo de 3 preguntas el bloque queda superado y se suman puntos de XP al perfil. Los puntos varían según el nivel: 20 XP en bloques fáciles, 30 XP en intermedios y 50 XP en difíciles. Al superar el bloque aparecen dos botones: "Volver a bloques" y "Pasar al bloque 2" para continuar directamente con el siguiente.

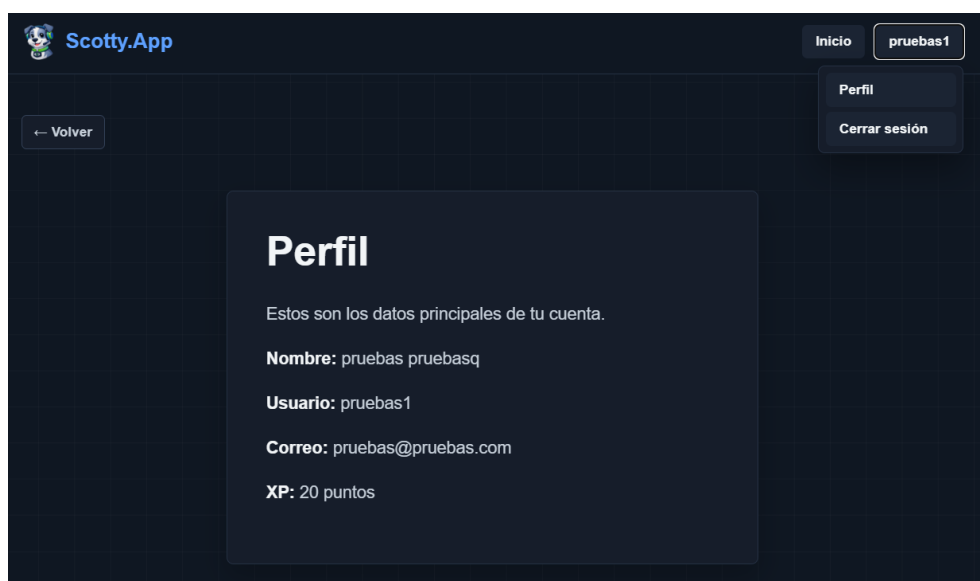
Figura 16 — Resultado bloque superado



- Perfil

Haciendo clic en el nombre de usuario aparece un menú desplegable con las opciones "Perfil" y "Cerrar sesión". En la pantalla de perfil se muestran el nombre completo, el nombre de usuario, el correo electrónico y el XP total acumulado.

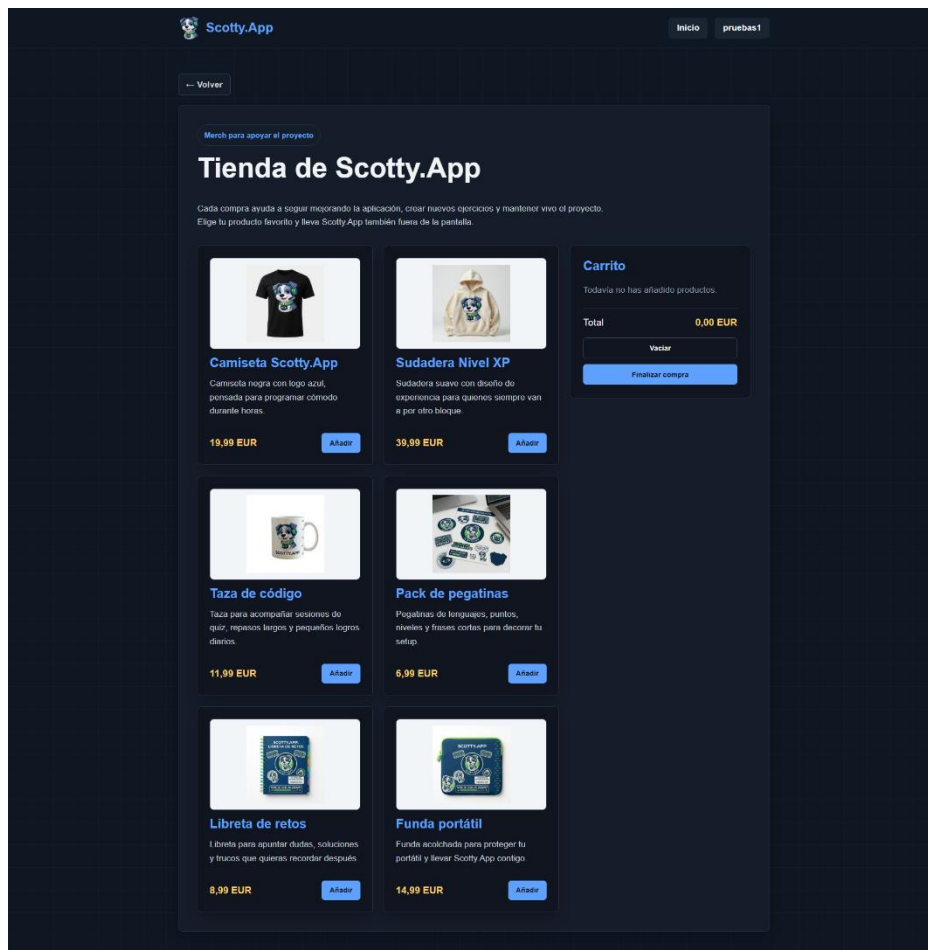
Figura 17 — Perfil del usuario



- Tienda

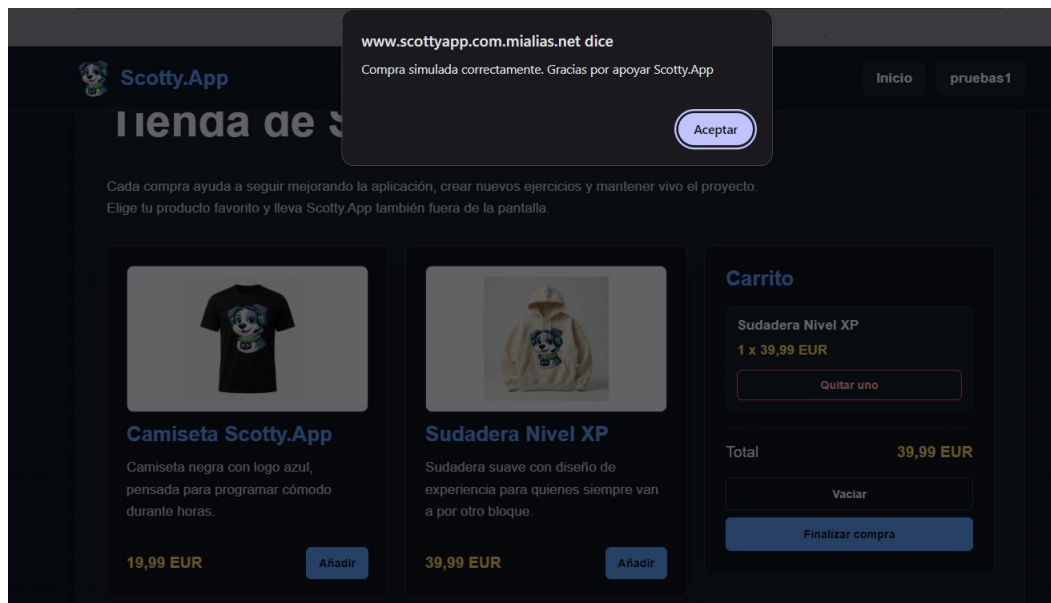
En la tienda se muestra el catálogo de productos con imagen, nombre, descripción y precio.

Figura 18 — Tienda de Scotty.App



Para añadir un producto al carrito se pulsa el botón "Añadir". El carrito aparece a la derecha y se actualiza en tiempo real mostrando los productos añadidos, la cantidad y el total. El botón "Quitar uno" reduce la cantidad de un producto, "Vaciar" limpia el carrito completo y "Finalizar compra" completa el proceso mostrando un mensaje de confirmación.

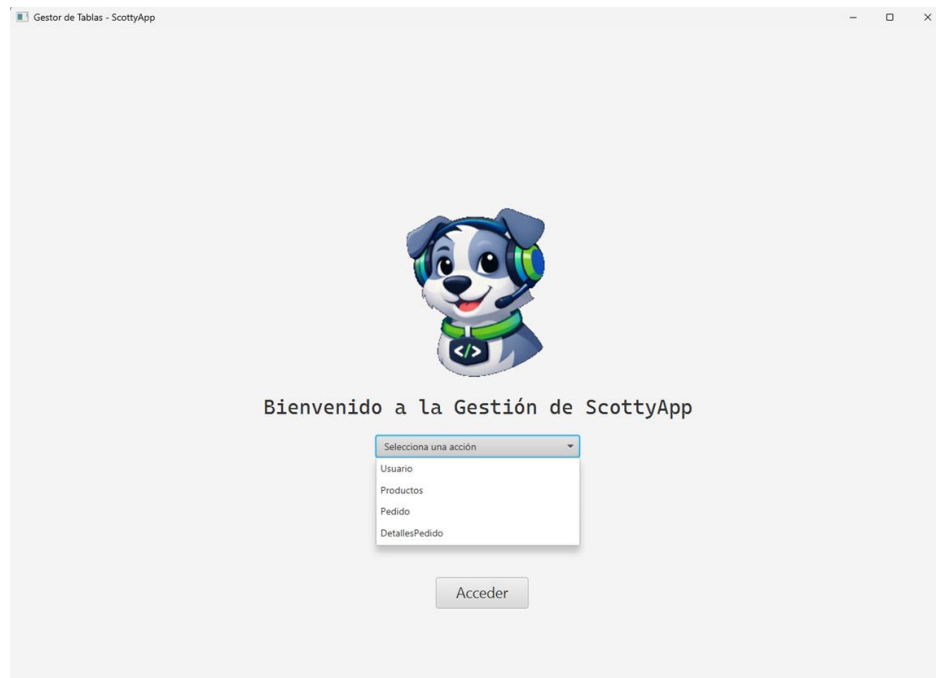
Figura 19 — Carrito con compra simulada



Aplicación administrativa JavaFX:

Al abrir la aplicación aparece la pantalla de bienvenida con un menú desplegable con cuatro opciones: Usuario, Productos, Pedido y DetallesPedido. Se selecciona la opción deseada y se pulsa "Acceder".

Figura 20 — Pantalla principal JavaFX



- Gestión de usuarios

Muestra una tabla con todos los usuarios registrados: id, nombre, email, contraseña, rol, dirección y teléfono. Desde aquí el administrador puede buscar usuarios escribiendo en el campo de búsqueda, crear uno nuevo con "Nuevo", seleccionar un usuario de la tabla para cargar sus datos en el formulario inferior, modificarlos y guardarlos con "Guardar", o eliminarlo con "Eliminar". El botón "Editar" permite modificar los campos del usuario seleccionado. Al terminar, "Volver al menú principal" regresa a la pantalla inicial.

Figura 21 — Gestión de usuarios JavaFX

The screenshot shows a JavaFX application window titled "Gestor de Tablas - ScottyApp". The window contains a table with user data and a form for managing users.

idUser	Name	Email	Password	Role	Address	Phone
2	Manuela Planelles	manuela@scottyapp.com	1234	ADMINISTRATOR	Avda. Denia 140	600000001
5	José Luis	joseluis@scottyapp.com	1234	ADMINISTRATOR	Calle Mayor 5	600000002
6	José Ramón	joseramon@scottyapp.com	1234	ADMINISTRATOR	Calle Sol 12	600000003
7	Iván	ivan@scottyapp.com	1234	ADMINISTRATOR	Calle Luna 8	600000004
8	Silvia	sdianezjuan@edu.gva.es	1234	ALUMNI	IES Mutxamel	600000005
9	Izan	izan@scottyapp.com	1234	ALUMNI	Calle Paz 3	600000006
10	Patricia	patricia@scottyapp.com	1234	ALUMNI	Calle Real 7	600000007
11	Ismael	ismael@scottyapp.com	1234	ALUMNI	Calle Norte 15	600000008
12	Alejandro	alejandro@scottyapp.com	1234	ALUMNI	Calle Sur 22	600000009
13	paula martinez diaz	man@ma.com	123456	ALUMNI	kjfbjkabckj	123456789

Below the table is a form for managing users. It includes a search bar with a "Buscar" button. To the right of the search bar are buttons for "Nuevo", "Editar", "Eliminar", and "Guardar". The form fields are: Id, Name, Role (dropdown), Password, Email, Phone, and Address. At the bottom left is a button labeled "<-- Volver al menú principal".

- Gestión de productos

Muestra el catálogo completo de productos con id, nombre, descripción, precio, stock y categoría. Funciona igual que la gestión de usuarios: se puede buscar, crear, editar, guardar y eliminar productos. El stock se actualiza automáticamente cuando se procesa un pedido gracias al trigger definido en la base de datos.

Figura 22 — Gestión de productos JavaFX

The screenshot shows a JavaFX application window titled "Gestor de Tablas - ScottyApp". It features a table with 6 columns: ID, Name, Description, Price, Stock, and Category. The table lists 16 products, including various computer peripherals, hardware components, and merchandise. Below the table is a form for managing products, with fields for Id, Name, Description, Price, and Stock, and a dropdown for Category. Action buttons include "Nuevo", "Editar", "Eliminar", and "Guardar". A search bar with a "Buscar" button is located above the form. A button at the bottom left says "<-- Volver al menú principal".

ID	Name	Description	Price	Stock	Category
1	Teclado Mecánico	Teclado mecánico RGB switches blue	59.99	25	Peripherals
2	Ratón Gaming	Ratón óptico 12000 DPI RGB	39.99	40	Peripherals
3	Monitor 24"	Monitor Full HD 144Hz IPS	189.99	15	Hardware
4	SSD 1TB	Disco sólido NVMe PCIe 4.0	89.99	30	Hardware
5	Auriculares	Auriculares gaming 7.1 surround	49.99	20	Peripherals
6	Tarjeta Gráfica	GPU 8GB GDDR6 Ray Tracing	399.99	8	Hardware
7	RAM 16GB	Memoria DDR5 5200MHz	69.99	35	Hardware
8	Alfombrilla XL	Alfombrilla gaming 900x400mm	19.99	50	Accessories
9	Webcam HD	Cámara 1080p 60fps con micrófono	79.99	18	Peripherals
10	Hub USB-C	Hub 7 en 1 USB-C HDMI 4K	34.99	45	Accessories
11	Camiseta Scotty.App	Camiseta negra con logo azul, pensada para programar cómodo dura...	19.99	50	Merchandising
12	Sudadera Nivel XP	Sudadera suave con diseño de experiencia para quienes siempre van ...	39.99	30	Merchandising
13	Taza de código	Taza para acompañar sesiones de quiz, repasos largos y pequeños lo...	11.99	100	Merchandising
14	Pack de pegatinas	Pegatinas de lenguajes, puntos, niveles y frases cortas para decorar t...	6.99	200	Merchandising
15	Libreta de retos	Libreta para apuntar dudas, soluciones y trucos que quieras recordar ...	8.99	75	Merchandising
16	Funda portátil	Funda acolchada para proteger tu portátil y llevar Scotty.App contigo.	14.99	40	Merchandising

- Gestión de pedidos

Muestra todos los pedidos con su id, fecha, total, estado e id de usuario. Los estados posibles son PENDING, PROCESSED y CANCELLED. Permite buscar pedidos por estado o usuario, crear nuevos, editarlos y eliminarlos. En la parte inferior aparece la lista de productos disponibles con su precio para añadirlos al pedido indicando la cantidad con el botón "Añadir producto". El botón "Ver detalles" muestra el desglose de productos de cada pedido.

Figura 23 — Gestión de pedidos JavaFX

The screenshot displays the 'Gestor de Tablas - ScottyApp' window. It features a table with the following data:

ID Pedido	Fecha	Total	Estado	ID Usuario
1	2026-01-15	59.98	PROCESSED	2
2	2026-02-03	189.99	PROCESSED	5
3	2026-02-20	49.99	PENDING	6
4	2026-03-10	399.99	PENDING	7
5	2026-03-25	109.97	CANCELLED	8
6	2026-04-05	89.99	PROCESSED	9
7	2026-04-18	26.98	PENDING	10
8	2026-05-01	469.98	PENDING	2

Below the table, there is a search bar labeled 'Buscar por estado o usuario' with a 'Buscar' button. To the right are buttons for 'Editar', 'Eliminar', 'Ver detalles', and 'Nuevo'. Below these are input fields for 'ID', 'Fecha', a 'Selecciona usuario' dropdown, a 'Status' dropdown, and a 'Total' input field, followed by a 'Guardar' button.

The 'Productos disponibles:' section contains a table with 7 items:

ID	Nombre	Precio
1	Teclado Mecánico	59.99
2	Ratón Gaming	39.99
3	Monitor 24"	189.99
4	SSD 1TB	89.99
5	Auriculares	49.99
6	Tarjeta Gráfica	399.99
7	RAM 16GB	69.99

To the right of this table is a 'Cantidad:' section with a 'Cant.' input field and an 'Añadir producto' button. At the bottom left, there is a '<-- Volver al menu principal' button.

Scotty App está pensada para que cualquier persona pueda empezar a practicar programación en pocos pasos, sin instalaciones ni configuraciones complejas. Desde la landing page hasta el último bloque de quizzes, todo el flujo está diseñado para ser intuitivo y directo. La aplicación administrativa complementa la plataforma dando al equipo las herramientas necesarias para mantener el sistema actualizado y gestionar los pedidos de la tienda.

8. Requisitos e instalación

8.1 Requisitos del sistema

Para usar Scotty App no hace falta ninguna instalación especial si eres alumno. Si eres administrador, sí necesitas tener un par de cosas instaladas en tu equipo. La siguiente tabla resume lo que necesita cada perfil:

Componente	Requisito plataforma web (alumno/usuario)	Requisitos Aplicación administrativa (Administrador)
Sistema operativo	Cualquier SO con navegador moderno	Windows 10 o superior de 64 bits
Navegador	Chrome 90+, Firefox 88+, Edge 90+ o similar	No requiere navegador
JavaScript	Debe estar activado en el navegador	No aplica
Conexión a internet	Requerida	No requerida
Procesador	Sin requisito específico	Compatible con arquitectura de 64 bits
RAM	Sin requisito específico	4 GB recomendados para un funcionamiento fluido
Espacio en disco	Sin requisito específico	500 MB libres mínimo
Java	No requiere instalación	JDK 21 LTS o superior
Base de datos	No requiere instalación	MariaDB en ejecución
Resolución de pantalla	1024 x 768 px o superior	1280 x 720 px o superior

La parte web funciona directamente desde el navegador sin necesidad de instalar nada, ya que toda la lógica se ejecuta en el cliente mediante JavaScript. La aplicación administrativa en cambio necesita Java instalado en el equipo porque es el motor que la hace funcionar, y MariaDB porque es donde se almacenan los datos de usuarios, productos y pedidos. La conexión entre ambos se realiza mediante el driver JDBC de MariaDB, que ya viene incluido en el proyecto.

8.2 Instalación y ejecución

Plataforma web

No necesita instalación. El alumno accede directamente desde el navegador a través de la URL del proyecto. Si se quiere probar en local, basta con clonar el repositorio desde GitHub y abrir el archivo index.html en el navegador.

Aplicación administrativa JavaFX

La aplicación se entrega como un archivo .jar que ya lleva todas las dependencias incluidas. Distinguimos dos momentos: la instalación inicial, que se hace una sola vez, y el uso diario.

Instalacion inicial:

1. Instalar Java JDK 21 LTS o superior: es el motor que necesita la aplicación para ejecutarse. Se puede descargar desde la web de Eclipse Adoptium. Es importante que Java quede registrado en el PATH del sistema durante la instalación.
2. Instalar MariaDB: es el sistema gestor de base de datos donde se almacena toda la información. Se puede instalar de forma independiente o a través de XAMPP.
3. Crear la base de datos: con MariaDB en marcha, abrir HeidiSQL, conectarse al servidor local y ejecutar el script BD_ScottyApp.sql incluido en el repositorio. Este script crea automáticamente la base de datos con todas sus tablas, claves foráneas, trigger y stored procedure.
4. Crear el acceso directo: copiar el archivo ScottyApp-1.0-SNAPSHOT.jar en una carpeta del equipo y crear en esa misma carpeta un archivo ScottyApp.bat con el siguiente contenido:

```
bat
@echo off
java -jar ScottyApp-1.0-SNAPSHOT.jar
```

Crear un acceso directo del .bat en el escritorio.

Uso diario.

Una vez instalado, para usar la aplicación cada día solo hay que hacer dos cosas:

1. Arrancar MariaDB desde XAMPP o desde el servicio del sistema.
2. Hacer doble clic en el acceso directo del escritorio.

Si MariaDB no está en ejecución cuando se abre la aplicación, la conexión fallará y no se podrá acceder al sistema.

9. Plan de negocio

9.1 Público objetivo

Queremos dirigirnos a un público joven y en formación dentro del ámbito tecnológico. Lo hemos dividido en tres niveles:

Nuestro público principal son estudiantes de ciclos formativos y carreras tecnológicas de entre 18 y 30 años que necesitan reforzar la práctica de programación. Usuarios con conocimientos básicos que buscan mejorar su lógica y fluidez de forma dinámica, como nosotros mismos.

A medio y largo plazo nos gustaría llegar también a personas en proceso de cambio profesional y empresas tecnológicas interesadas en formar o evaluar habilidades técnicas de forma práctica.

Queremos centrarnos en estudiantes que ya han tenido contacto con la programación pero carecen de práctica constante, posicionando Scotty App como una herramienta de refuerzo más que de aprendizaje teórico desde cero.

9.2 Utilidad social o empresarial

Nuestro objetivo es mejorar la forma en la que se practica la programación, aportando valor tanto a nivel educativo como profesional.

Desde el punto de vista educativo, la plataforma se posiciona como una herramienta de refuerzo práctico que complementa el aprendizaje teórico tradicional. Su enfoque en la repetición, el feedback inmediato y la gamificación favorece la constancia del usuario y mejora la adquisición de habilidades técnicas.

Desde el punto de vista empresarial, Scotty App tiene potencial para crecer dentro del sector de las plataformas educativas digitales, tanto con usuarios individuales como con centros educativos y academias. Su modelo permite evolucionar desde una plataforma gratuita hacia servicios de pago y colaboraciones con entidades formativas o empresas tecnológicas.

9.3 Costes y beneficios

Para una primera versión funcional y escalable, se estiman los siguientes costes (MVP realista):

- Dominio web: 10 €/año
- Hosting backend: 5–20 €/mes
- Base de datos: 10–30 €/mes

- Marketing inicial: 0–200 €

El coste mensual estimado se sitúa entre 20 y 60 €/mes, ajustable según el crecimiento de usuarios.

De cara al futuro, nos gustaría explorar un modelo de ingresos mixto basado en varias vías:

- Acceso gratuito con extras de pago: la base de la plataforma siempre será gratuita, pero con opción a contenido premium como niveles avanzados o funcionalidades extra.
- Tienda de productos: compra de productos de informática y merchandising de la marca dentro de la plataforma.
- Licencias para centros educativos: a medio plazo, ofrecer herramientas de seguimiento y gestión de alumnos a academias y centros formativos.
- Patrocinios y colaboraciones: empresas tecnológicas interesadas en visibilidad o en captar talento entre nuestros usuarios.

Para medir si el proyecto va por buen camino, nos fijaríamos en una serie de indicadores clave: el número de usuarios registrados como primer indicador de alcance, los usuarios activos cada mes para entender el uso real más allá del primer acceso, la tasa de retención para saber si los usuarios encuentran valor suficiente para volver, los ingresos por usuario para entender cuánto valor genera cada uno dentro del sistema, y la conversión de usuarios gratuitos a de pago, que nos indicaría si el modelo es sostenible a largo plazo.

10. Prevención de riesgos laborales

Desarrollamos el proyecto de forma telemática, cada uno desde casa. Aunque pueda parecer un entorno seguro, el trabajo continuado con pantallas conlleva riesgos reales que conviene conocer y gestionar.

La normativa que regula estas condiciones es la Ley 31/1995 de Prevención de Riesgos Laborales y el Real Decreto 488/1997, específico para trabajos con pantallas de visualización de datos.

10.1 Identificación de riesgos

Riesgos derivados del espacio de trabajo:

Trabajar desde casa implica no siempre disponer de una iluminación adecuada. Según el RD 486/1997, en oficinas y aulas la iluminación mínima recomendada es de 500 lux. Los reflejos en la pantalla y una luz insuficiente pueden provocar fatiga visual, escozor de ojos, dolores de cabeza y dificultades de atención.

Riesgos ergonómicos (Carga física):

El uso de mobiliario doméstico no adaptado, como mesas bajas o sillas sin regulación, puede derivar en trastornos musculoesqueléticos: lumbalgias, cervicalgias, dolores de cabeza y problemas circulatorios en las piernas por permanecer mucho tiempo sentados. Trabajar con el ordenador de forma prolongada también puede causar problemas oculares como sequedad, enrojecimiento y fatiga visual.

Riesgos ergonómicos (Postura):

Mantener una postura incorrecta frente al ordenador durante horas es uno de los riesgos más habituales en nuestro sector. Los trastornos musculoesqueléticos más frecuentes se producen en el cuello, los hombros y la zona dorsolumbar.

Riesgos psicosociales (Carga mental):

El trabajo a distancia puede generar estrés por el cumplimiento de los hitos y dificultar la separación entre la vida personal y la laboral. Una carga mental excesiva puede derivar en estrés, fatiga mental, insomnio, dificultad de concentración e incluso burnout. La falta de comunicación presencial y el aislamiento también pueden afectar negativamente al bienestar del equipo.

Riesgos derivados de agentes físicos:

Las pantallas emiten radiaciones no ionizantes que, aunque menos peligrosas que las ionizantes, pueden ocasionar fatiga ocular y dolores de cabeza con una exposición prolongada sin descansos.

10.2 Medidas preventivas

Higiene postural: mantenemos la espalda recta y apoyada en el respaldo, la pantalla a la altura de los ojos o ligeramente por debajo, los antebrazos apoyados formando un ángulo de 90°, y los pies apoyados en el suelo o sobre un reposapiés. El ordenador se coloca perpendicular a la ventana para evitar reflejos y deslumbramientos.

Iluminación adecuada: intentamos trabajar con luz natural siempre que sea posible, complementándola con iluminación artificial uniforme que no produzca brillos ni reflejos sobre la pantalla.

Protección ocular: usamos filtros de luz azul para reducir la fatiga visual en sesiones largas de trabajo y mantenemos una distancia adecuada a la pantalla de entre 40 y 60 cm.

Pausas activas: establecemos descansos de 5 a 10 minutos por cada hora de trabajo para estirarnos, relajar la vista y cambiar de postura, reduciendo así la fatiga acumulada.

Gestión de la carga mental: distribuimos el trabajo a lo largo del hito para evitar concentrar todo el esfuerzo en los últimos días. La comunicación constante entre el equipo a través de Discord nos ayuda a detectar bloqueos y redistribuir la carga cuando alguien lo necesita, reduciendo el estrés individual.

11. Integración de los módulos del ciclo en el proyecto

11.1 Lenguajes de marcas

En este módulo hemos aplicado HTML5 para estructurar el contenido de la plataforma, CSS3 para el diseño visual y la adaptación a distintos dispositivos, y JavaScript para la lógica de los quizzes, la validación de formularios y la gestión del perfil del usuario mediante localStorage.

11.2 Programación

Hemos desarrollado la aplicación administrativa en Java utilizando JavaFX como framework de interfaz gráfica y SceneBuilder para el diseño visual de las pantallas. La conexión con la base de datos se realiza mediante el conector JDBC de MariaDB, gestionando la base de datos desde HeidiSQL. Como entorno de desarrollo hemos utilizado IntelliJ IDEA y SourceTree para el control de versiones.

11.3 Bases de datos

n el módulo: el diagrama Entidad-Relación en notación de Chen, el modelo relacional derivado del E-R, y el script SQL de creación de tablas en SQL Server incluyendo checks, valores por defecto y claves foráneas. El script se ha adaptado también a MariaDB para su uso real en la aplicación JavaFX. Además, hemos implementado un trigger que actualiza automáticamente el stock de los productos cuando el administrador procesa un pedido desde la aplicación, garantizando la integridad de los datos desde la propia base de datos.

11.4 Entornos de desarrollo

Hemos aplicado los conocimientos del módulo en varias áreas del proyecto: la definición del objetivo mediante el enfoque SMART, el análisis y diseño con los diagramas de arquitectura, casos de uso y clases, la planificación del proyecto con Scrum y el tablero

Kanban de Trello, el control de versiones con GitHub, y las pruebas de la aplicación aplicando técnicas de caja negra y análisis de valores límite

11.5 Sistemas Informáticos

Hemos aplicado los conocimientos del módulo en la definición de los requisitos hardware y software necesarios para ejecutar cada componente del proyecto, evaluando las necesidades del sistema en relación con su desarrollo y puesta en marcha. Identificamos los requisitos específicos tanto para el alumno, que solo necesita un navegador moderno, como para el administrador, que requiere un sistema de 64 bits, Java JDK 21 y MariaDB en ejecución.

11.6. Inglés

El proyecto incorpora el inglés en varios niveles: el abstract de presentación está redactado íntegramente en inglés, la nomenclatura de la base de datos sigue convenciones en inglés, y el vídeo de presentación del proyecto se realizará también en inglés.

11.7. IPE I

Hemos aplicado los contenidos del módulo en el apartado de Prevención de Riesgos Laborales, identificando los riesgos asociados al trabajo con pantallas en un entorno de teletrabajo y definiendo las medidas preventivas correspondientes según la normativa vigente.

12. Conclusiones

12.1 Conclusiones sobre el trabajo realizado

Scotty App ha sido un proyecto ambicioso que hemos ido ajustando a lo largo del curso hasta encontrar un enfoque realista y coherente. Empezamos con una idea más compleja que incluía economía virtual, tokens y un sistema de canjes, y fuimos simplificando hasta llegar a lo que realmente podíamos construir y defender: una plataforma de refuerzo práctico basada en quizzes gamificados con una tienda complementaria y una aplicación administrativa en JavaFX.

El resultado final es un proyecto que funciona, tiene sentido y es defendible. El frontend está operativo y publicado, la base de datos está diseñada e implementada, y la aplicación JavaFX está desarrollada y conectada a la base de datos de forma independiente. No hemos intentado aparentar más de lo que es, y creemos que eso es precisamente lo que le da solidez.

Algunas partes del proyecto quedaron planteadas a nivel arquitectónico pero no completamente integradas, especialmente la sincronización entre el frontend web y la base de datos. Esta limitación, debida al tiempo disponible y al alcance del temario trabajado, queda recogida como línea de mejora futura en el apartado 12.3.

12.2 Aprendizaje adquirido

Este proyecto nos ha permitido aplicar de forma práctica casi todos los conocimientos adquiridos durante el curso: HTML, CSS y JavaScript para el frontend, Java y JavaFX para la aplicación administrativa, SQL Server para el diseño y documentación de la base de datos, y MariaDB para su implementación real en la aplicación JavaFX, y Scrum para la organización del equipo. Más allá de lo técnico, hemos aprendido a trabajar en equipo, a gestionar imprevistos como la salida de un miembro del grupo, y a tomar decisiones de diseño justificadas.

12.3 Posibles ampliaciones y mejoras

De cara a una segunda versión de Scotty App, hemos identificado las siguientes líneas de mejora:

Persistencia completa mediante backend web: actualmente los datos del alumno se gestionan con localStorage, lo que implica que se pierden si el usuario cambia de dispositivo o borra el almacenamiento del navegador. La siguiente versión incorporaría un backend real en Java con una API REST que conecte el frontend directamente con la base de datos, garantizando la persistencia real de todos los datos.

Autenticación segura: el sistema de login actual es funcional pero básico. En una versión real las contraseñas se almacenarían cifradas en el servidor y el sistema de sesiones se gestionaría mediante tokens seguros, eliminando la dependencia de localStorage para la autenticación.

Quizzes dinámicos desde base de datos: actualmente los quizzes están hardcodeados en el HTML. En la siguiente versión se gestionarían desde JavaFX y se servirían dinámicamente desde la base de datos, permitiendo al administrador crear, editar y eliminar preguntas sin tocar el código.

Pasarela de pago: la tienda actual muestra el catálogo de productos pero no permite realizar compras reales. La integración de una pasarela de pago permitiría completar el flujo de compra y conectar los pedidos del frontend con la gestión de JavaFX.

Sincronización frontend-JavaFX: actualmente ambas partes funcionan de forma independiente. En una versión futura estarían conectadas a través del backend, de forma que los cambios realizados desde JavaFX se reflejarían automáticamente en la plataforma web.

Micropagos y personalización: en una versión futura se podría incorporar un sistema de micropagos que permitiera al usuario personalizar su avatar o desbloquear ventajas dentro de la plataforma.

13. Bibliografía y webgrafía

Entornos de Desarrollo

Diáñez Juan, S. (Curso 2025/2026). UP 1 — Desarrollo del software. Fases del ciclo de vida: análisis, diseño, codificación, pruebas, documentación, explotación y mantenimiento. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). UP 4 — Optimización y documentación. Control de versiones y repositorios. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). UP 5 — Elaboración de diagramas de clases. UML, relaciones, herencia, visibilidad. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). UP 6 — Elaboración de diagramas de comportamiento. Diagramas de casos de uso, actores, relaciones «include» y «extend». IES Mutxamel.

Schwaber, K., y Sutherland, J. (2020). La Guía de Scrum. Las reglas del juego (versión noviembre 2020). Scrum.org. Recuperado de <https://scrumguides.org>

JUnit Team. (2026). Documentación oficial de JUnit 5. Recuperado de <https://junit.org/junit5>

Sistemas Informáticos

Vera Lloret, L. J. (Curso 2025/2026). Sistemas Informáticos (SIN). IES Mutxamel.

Bases de Datos

Fraile Antón, M. V. (Curso 2025/2026). Módulo Bases de Datos — RA6: ER Model y Design an ER Diagram. RA6: Relational Model. RA4: Data Definition Language. Script SQL de creación de tablas. IES Mutxamel.

Programación

Benavente Domenech, P. (Curso 2025/2026). Módulo Programación. Programación orientada a objetos en Java, acceso a datos y programación gráfica. IES Mutxamel.

Oracle Corporation. (2026). Documentación oficial de Java. Recuperado de <https://docs.oracle.com/en/java>

Gluon. (2026). Documentación oficial de JavaFX. Recuperado de <https://openjfx.io>

MariaDB Foundation. (2026). Documentación oficial de MariaDB. Recuperado de <https://mariadb.org/documentation>

HeidiSQL. (2026). Herramienta de gestión de bases de datos. Recuperado de <https://www.heidisql.com>

Atlassian. (2026). SourceTree. Cliente visual para gestión de repositorios Git. Recuperado de <https://www.sourcetreeapp.com>

GitHub Inc. (2026). Documentación oficial de GitHub. Recuperado de <https://docs.github.com>

Lenguajes de Marcas

López Coloma, F. (Curso 2025/2026). Módulo Lenguaje de Marcas — RA2: HTML y CSS. RA3: JavaScript. IES Mutxamel.

W3Schools. (2026). Tutoriales de HTML5, CSS3 y JavaScript. Recuperado de <https://www.w3schools.com>

Itinerario Personal para la Empleabilidad (IPE)

Caldas Blanco, M. E., Hidalgo Ortega, M. L., y Laraño Díaz, J. E. (2024). Itinerario Personal para la Empleabilidad I (ISBN: 978-84-1134-936-9). Editorial Editex, S. A.

Ley 31/1995, de 8 de noviembre, de Prevención de Riesgos Laborales. Boletín Oficial del Estado, núm. 269, de 10 de noviembre de 1995.

Real Decreto 488/1997, de 14 de abril, sobre disposiciones mínimas de seguridad y salud relativas al trabajo con equipos que incluyen pantallas de visualización. Boletín Oficial del Estado, núm. 97, de 23 de abril de 1997.

Real Decreto 486/1997, de 14 de abril, sobre disposiciones mínimas de seguridad y salud en los lugares de trabajo. Boletín Oficial del Estado, núm. 97, de 23 de abril de 1997.

Benchmarking y referencias

Kahoot! (2026). Plataforma de aprendizaje gamificado. Recuperado de <https://kahoot.com>

Quizizz. (2026). Plataforma de quizzes educativos. Recuperado de <https://quizizz.com>

CodeCombat. (2026). Plataforma de aprendizaje de programación mediante videojuegos. Recuperado de <https://codecombat.com>

OpenAI. (2026). ChatGPT. Modelo de inteligencia artificial utilizado como apoyo en la redacción y generación de imágenes del proyecto. Recuperado de <https://chat.openai.com>

Anthropic. (2026). Claude. Modelo de inteligencia artificial utilizado como apoyo en el análisis y revisión de coherencia documental. Recuperado de <https://claude.ai>

Atlassian. (2026). Trello. Herramienta de gestión de tareas. Recuperado de <https://trello.com>.